
Uncovering Competency Gaps in Large Language Models and Their Benchmarks

Maty Bohacek^{1 2} Nino Scherrer³ Nicholas Dufour² Thomas Leung² Chris Bregler² Stephanie C.Y. Chan²

Abstract

The evaluation of large language models relies heavily on standardized benchmarks. These benchmarks provide useful aggregated metrics, but can obscure (i) particular sub-areas where the models are weak (“model gaps”) and (ii) imbalanced coverage in the benchmarks themselves (“benchmark gaps”). To automatically uncover both types of gaps, we propose a simple new method using concept activations from sparse autoencoders, to identify fine-grained gaps on a per-concept basis. The method also benefits from grounding evaluation in the model’s internal representations, as well as easy comparison across benchmarks. We applied the method to five popular open-source models and over a dozen benchmarks, as illustrative examples. As validation of the approach, we found that our automatic, unsupervised method was able to recover model gaps that have been previously documented in the literature (e.g., sycophancy), in addition to identifying novel model gaps. We were also able to automatically uncover benchmark gaps: core concepts that should fall within the scope of a given benchmark. Our “competency gaps” method can complement existing benchmarks by providing a concept-level decomposition of model behavior, and can also help benchmark developers improve and iterate upon benchmark design. Code is available at [competency-gaps.github.io](https://github.com/competency-gaps).

1. Introduction

Evaluating large language models (LLMs) relies heavily on benchmarks that report *aggregated* scores (e.g., accuracy or pass@k). Over the last decade, hundreds of benchmarks

have been introduced across diverse domains (Guo et al., 2023; Chang et al., 2024). While these benchmarks have fueled progress, uniform aggregation can obscure important sub-trends and mask specific model weaknesses (Hardt, 2025; Burnell et al., 2023). For instance, Didolkar et al. (2024) disaggregated performance on MATH (Hendrycks et al., 2021) and found topic-wise scores ranging from 27% to 74%, despite an overall score of 54%.

To counteract these aggregation issues, some benchmarks provide “semantic” topic annotations (e.g., hand-curated topics in MATH (Hendrycks et al., 2021) or GPQA (Rein et al., 2024), or embedding-based clusters (Perez et al., 2023)). These high-level labels help characterize benchmark distributions and disaggregate performance, but they generally provide only coarse-grained insights into model strengths and weaknesses. We currently lack a view of how finer-grained concepts, contexts, and reasoning patterns extend beyond coarse topic labels, and how they relate to real-world model usage and capabilities (Miller & Tang, 2025; Mizrahi et al., 2024). Furthermore, many of these semantic annotations are manually curated and difficult to scale. Without a scalable, fine-grained understanding of benchmark distributions, we risk overlooking benchmark gaps, while systematically overtesting certain concepts.

In this work, we are interested in the automated identification of two types of gaps: (i) model gaps, i.e., concept domains where models systematically underperform, and (ii) benchmark gaps, i.e., concept domains that are inadequately represented in an evaluation dataset (see Figure 1). To this end, we introduce a simple new method called Competency Gaps (CG), which is both automatic and easy to scale. The method leverages sparse autoencoders (SAEs), which decompose the internal representations of an LLM into sparse, interpretable feature vectors called SAE concept activations (Bricken et al., 2023; Cunningham et al., 2023). Each dimension of these vectors is trained to capture a distinct concept, assuming a sufficiently diverse and representative training distribution. Autointerpretability methods (McGrath et al., 2024) can be applied to assign each dimension (or “concept”) a textual label. This fixed set of concepts is often referred to as a concept dictionary, with the dictionary size often numbering thousands or millions

¹Stanford University ²Google DeepMind ³Google, Paradigms of Intelligence Team. Correspondence to: Maty Bohacek <maty@stanford.edu>.

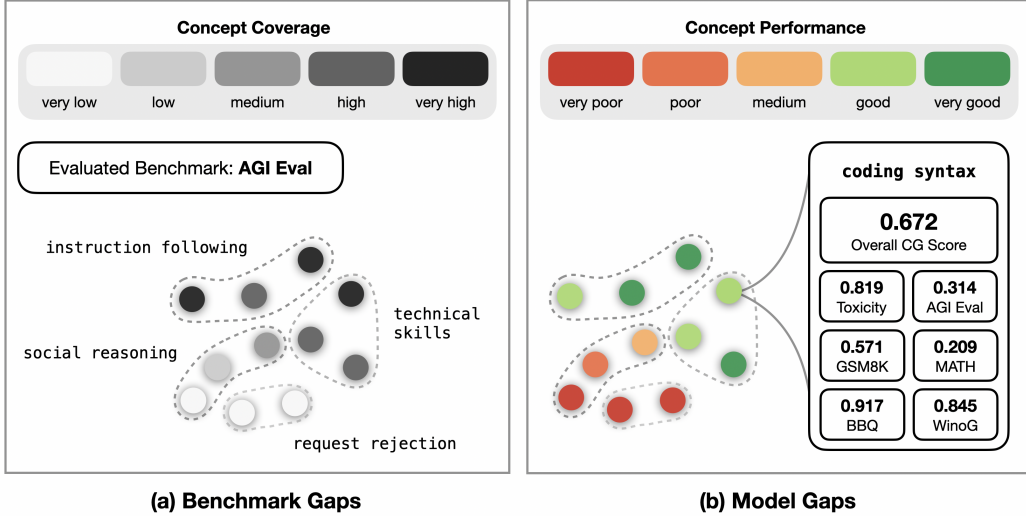


Figure 1. Method Overview. The method decomposes LLM evaluation into fine-grained, interpretable “competency gaps”, using the concept dictionary learned by a sparse autoencoder. The figure provides a simplified illustration using a small number of concepts (each circle is one concept). *(a) Benchmark Gaps* surface underrepresented concepts for a given benchmark (e.g. AGI Eval). We can also group the concepts into larger concept domains, e.g. “social reasoning”. *(b) Model Gaps* surface specific concept areas where a model may be under-performing. Each concept (e.g. “coding syntax”) can be evaluated for an individual benchmark (e.g. GSM8K) or for a suite of benchmarks (“Overall CG Score”).

of concepts. In our method, we analyze the benchmark distribution over the SAE concept dictionary to identify benchmark gaps, and also map the model’s performance onto the concept space to identify model gaps¹.

As a demonstration of the introduced method, we evaluate Competency Gaps (CG) on five popular open-source models (Gemma2-2B-Instruct (Team et al., 2024), Llama3.1-8B-Instruct (Grattafiori et al., 2024), Mistral-7B-Instruct-v0.1 (Jiang et al., 2023), Qwen3-4B (Yang et al., 2025), and DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025)) across a broad range of benchmarks — ten in our main suite, complemented by an arena-style benchmark (LM SYS Chatbot Arena) and additional capability benchmarks (SWE-Bench, Terminal-Bench, MMLU, GPQA, HLE).

- **Competency Gaps (CG) Method.** We introduce a systematic method for the automated identification of benchmark coverage and model performance gaps, using an SAE-based approach. The method can be applied to any LLM and benchmark of interest.² The method decomposes aggregated benchmark scores into a large number of interpretable axes derived from the model’s

¹The proposed approach can only detect benchmark and model gaps for concepts that are represented in the SAE space. Concept coverage of CG hence improves with more representative SAEs.

²Using an SAE trained specifically for the model of interest allows the analysis to be grounded in that model’s own representations. However, we demonstrate that even LLMs without a dedicated SAE can be evaluated using CG by leveraging the SAE from another model (Section 4.2.3).

own representations, surfaces patterns of over- and under-tested concepts, and identifies actionable improvements for both models and benchmarks, as described in Figure 2. The implementation is fully open sourced.

- **Demonstrations of the Method on Popular LLMs and Benchmarks.** We applied CG to five popular open-source models (Gemma2-2B-Instruct, Llama3.1-8B-Instruct, Mistral-7B-Instruct-v0.1, Qwen3-4B, and DeepSeek-R1-Distill-Llama-8B) over more than a dozen diverse benchmarks, illustrating the kinds of insights the method enables.
- **Interactive Web Tool.** We also release an open-source web application (Figure 5) that enables users to explore model capabilities and benchmark balance, in a transparent and interpretable way.

2. Related Work

Weakness Identification in LLMs. Identification of LLM weaknesses has evolved from anecdotal analysis to more systematic frameworks that break down performance into specific components (Jones & Steinhardt, 2022). Among the first to do this were HarmBench (Mazeika et al., 2024) and garak (Derczynski et al., 2024), which established standardized evaluations of harmful behaviors. Various methods have introduced autoraters for this task: AutoDetect (Cheng et al., 2024), for example, used three LLM-powered agents to achieve a 50%+ weakness identification success rate.

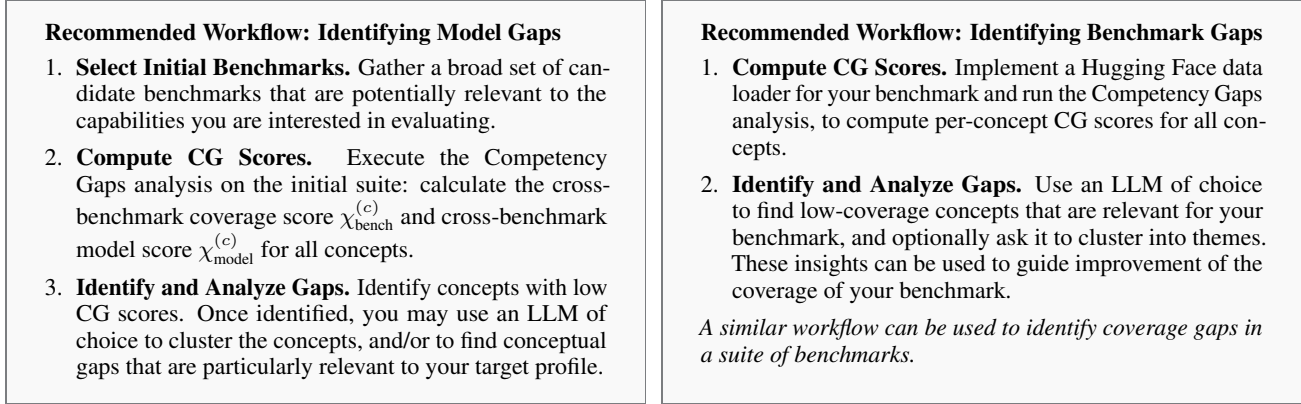


Figure 2. Recommended workflows for applying the Competency Gaps (CG) method.

More recently, EvalTree (Zeng et al., 2025) builds a hierarchical capability tree from benchmark instances and flags subtree nodes where the model performs below a statistical threshold. Gan et al. (2024) demonstrated that reasoning capabilities in LLMs can be systematically broken down and analyzed to identify specific weaknesses in logical inference chains. Other systematic evaluation methods have been proposed (Kim et al., 2023); some work in this domain has also taken an adversarial learning approach (Yang et al., 2024).

Cross-Model Comparisons. Going beyond simple benchmark scoreboards, recent work has explored approaches of characterizing the behavioral differences and internal representations across models and architectures in more detail (Zheng et al., 2025; Kim et al., 2025; Chang & Bergen, 2024). Some works, including BehaviorBox (Tjua et al., 2025) and the LLM Comparator (Kahng et al., 2024), have emphasized side-by-side comparisons and actionable insights. Some mechanistic interpretability methods have emerged as well, e.g. by patching activations between model locations and decoding representations into interpretable text (Nada et al., 2024). Recent work has also scrutinized the “universality hypothesis”, i.e., universal features and circuits should appear for similar tasks across architectures (Shu et al., 2025; Yin et al., 2024).

Analysis of Benchmark Quality. Automated quality detection has advanced through frameworks like CLEAR (Chen & Mueller, 2024) and SMART Filtering (Gupta et al., 2024), which automatically detect and filter problematic training data. A significant part of the work in this area has focused on bias evaluation (Yin et al., 2026; Manerba et al., 2023; Koo et al., 2023). Some preliminary work has also considered monitoring benchmark performance across time (Zhong & Raghunathan, 2025) or using meta-learning (Calian et al., 2025).

Comparison to Related Work. In Appendix L, we compare our method to some of the above-mentioned methods that

relate to the discovery of benchmark and model gaps. See L.1 for details of our comparison methodology, and L.2 and L.3 for a high-level comparison of approaches and features of each method. Appendices M, N, O, and P contain a detailed comparison of results from each method.

3. Competency Gaps (CG) Method

In this section, we introduce **Competency Gaps (CG)**, an automated, SAE-based method that can be used to systematically evaluate and identify:

- **Benchmark gaps** given a set of evaluation benchmarks $\mathcal{B}$. How good is the coverage of various concepts in a specific benchmark or set of benchmarks? Do the benchmarks have any concept coverage gaps?
- **Model gaps** given a language model M with an associated sparse autoencoder SAE . How well does the model perform across various concepts? What are its strengths and weaknesses?

Each benchmark $b \in \mathcal{B}$ is the triple $b = (X_b, Y_b, f_b)$ of inputs, references, and a scoring function; the associated dataset D_b is the concrete set of input–output pairs. Each concept $c \in C_{SAE}$ represents a distinct direction in the SAE space, and can be mapped to an autointerpretability label l_c (see Appendix A for a brief SAE primer). The SAE activation score $s_{c,i}$ quantifies how much the model activates concept c within a token sequence x_i .

SAE Concept Activation Score. A data point i from a benchmark consists of an input token sequence x_i . The concept c ’s activation on token $x_{i,j} \in x_i$ is computed via $SAE(x_{i,j}, c)$. We sum $SAE(x_{i,j}, c)$ over all tokens in x_i and divide by $|x_i|$ to obtain a per-token average, the **SAE activation score** $\tilde{s}_{c,i}$:

$$\tilde{s}_{c,i} = \frac{1}{|x_i|} \sum_j SAE(x_{i,j}, c) \quad (1)$$

The per-token average prevents longer data points from dominating downstream statistics simply by being verbose.

We describe our metrics in detail below. They satisfy the following: (i) all included benchmarks have an equal weight for computing the concept’s cross-benchmark CG score, regardless of their size; (ii) all data points have an equal weight for computing per-benchmark metrics, regardless of their token length. The SAE dictionary size $K = |C_{SAE}|$ is not tuned by us; it is fixed by the pre-trained SAE’s expansion factor (see Appendix A). Running the SAE adds the computational cost of approximately one extra projection at a single layer, which comes out to be a small fraction of the LLM’s overall computational demands for a single forward pass. CG is hence inexpensive relative to running the benchmarks themselves.

3.1. Benchmark Gaps

We now introduce a measure to quantify concept coverage within and across benchmarks, based on the normalized SAE concept activation score $\tilde{s}_{c,i}$ introduced above.

Per-Benchmark Coverage. To evaluate the coverage of a concept c in benchmark b , we define:

$$\chi_{\text{bench}}^{(b,c)} = \frac{\sum_{i \in D_b} \tilde{s}_{c,i}}{\frac{1}{|C_{SAE}|} \sum_{c' \in C_{SAE}} \sum_{i \in D_b} \tilde{s}_{c',i}} \quad (2)$$

which relates the activation of SAE concept c in dataset b to the average concept activation in b . Normalizing by the mean concept activation makes $\chi_{\text{bench}}^{(b,c)}$ scale-invariant across benchmarks, so values are directly comparable: $\chi_{\text{bench}}^{(b,c)} = 1$ corresponds to coverage at the benchmark’s own average.

Cross-Benchmark Coverage. The overall coverage for concept c in a benchmark suite $\mathcal{B}$ is the mean per-benchmark coverage taken over *all* benchmarks in $\mathcal{B}$ (benchmarks where c does not activate contribute 0 and pull the score down):

$$\mathbf{X}_{\text{bench}}^{(c)} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}_c} \chi_{\text{bench}}^{(b,c)} \quad (3)$$

Coverage Classification. We label a concept as *missing* from the benchmark suite if $\mathbf{X}_{\text{bench}}^{(c)} < \epsilon$ (we used e^{-5}). Among the remaining concepts, we consider the bottom decile (≤ 10 th percentile) to be *underrepresented*, and the top decile (≥ 90 th percentile) to be *overrepresented*. We can define underrepresented and overrepresented concepts

for each individual benchmark similarly by using the per-benchmark concept coverage $\chi_{\text{bench}}^{(b,c)}$.

3.2. Model Gaps

We now turn to the quantification of model gaps, and introduce a novel measure for model performance grounded in the SAE concept space.

Per-Benchmark Model Performance. To evaluate model performance on a concept c for a benchmark b , we define $\chi_{\text{model}}^{(b,c)}$:

$$\chi_{\text{model}}^{(b,c)} = \frac{\sum_{i \in D_b} m_b(i) \cdot \tilde{s}_{c,i}}{\sum_{i \in D_b} \tilde{s}_{c,i}} \quad (4)$$

where $m_b(i)$ is the performance of the model on datapoint i for benchmark b (normalized to $[0,1]$; higher is better). The scoring policy for $m_b(i)$ is defined by the benchmark. If no data points activate c in b , then $\chi_{\text{model}}^{(b,c)}$ is undefined.

Cross-Benchmark Model Performance. The overall performance for concept c in a benchmark suite $\mathcal{B}$ is the mean $\chi_{\text{model}}^{(b,c)}$ across all benchmarks:

$$\mathbf{X}_{\text{model}}^{(c)} = \frac{1}{|\mathcal{B}_c|} \sum_{b \in \mathcal{B}_c} \chi_{\text{model}}^{(b,c)} \quad (5)$$

We only consider benchmarks where $\chi_{\text{model}}^{(b,c)}$ is defined. If no data points in $\mathcal{B}$ activate concept c , then $\mathbf{X}_{\text{model}}^{(c)}$ is undefined.

Model Gap. We label a concept a *model gap* if $\mathbf{X}_{\text{model}}^{(c)} < \epsilon$, for the same small ϵ . These are concepts on which the model performs poorly.

4. Demonstrations of the Method

To demonstrate the kinds of insights enabled by our method, we report results of applying CG to five LLMs across several benchmark datasets. However, this analysis can be applied to any language model and is intended as part of an active, iterative process in which a benchmark or model can be continuously refined (see Figure 2).

4.1. Experimental Setup

Benchmarks. The presented method can be applied to any text-based benchmark. We demonstrate our method on ten static benchmark datasets that are regularly used for performance and safety evaluations, complemented by an

arena-style benchmark and an additional set of conventional capability benchmarks (the latter reported in Appendix H).

- *Factuality benchmarks*: Vectara (Hughes et al., 2023); Natural Questions (Kwiatkowski et al., 2019).
- *Math benchmarks*: GSM8K (Cobbe et al., 2021); MATH (Hendrycks et al., 2021).
- *Reasoning benchmarks*: AGI Eval (Zhong et al., 2023); LogicBench (Parmar et al., 2024); Social IQA (Sap et al., 2019); WinoGrande (Sakaguchi et al., 2021).
- *Ethics & bias benchmarks*: BBQ (Parrish et al., 2021); CROWS Pairs (Nangia et al., 2020).
- *Arena-style benchmark*: LMSYS Chatbot Arena (Zheng et al., 2023).
- *General ability benchmarks*: SWE-Bench (Jimenez et al., 2024); Terminal-Bench (Merrill et al., 2026); MMLU (Hendrycks et al., 2020); GPQA (Rein et al., 2024); Humanity’s Last Exam (HLE) (Phan et al., 2025).

Models. We analyzed five popular open-source LLMs with available SAEs (or transcoders) and autointerpretability labels. However, we emphasize that the method is not bound to these particular models; it can be applied to any LLM with an SAE.

- Llama3.1-8B-Instruct + Goodfire SAE attached at layer 19 (Grattafiori et al., 2024; Balsam et al., 2025).
- Gemma2-2B-Instruct + Gemma Scope SAE attached at layer 20 (residual stream) (Team et al., 2024; Lieberum et al., 2024);³
- Mistral-7B-Instruct-v0.1 + SAE attached at layer 16 (MLP output) (Jiang et al., 2023; Cosgrove, 2024).
- Qwen3-4B + transcoders on the residual stream (Yang et al., 2025; Hanna, 2024).
- DeepSeek-R1-Distill-Llama-8B + Llama-Scope-R1-Distill SAE attached at layer 20 (residual stream) (Guo et al., 2025; He et al., 2024).

The main paper reports results for Llama3.1-8B-Instruct on the ten static benchmarks (Section 4.2); results for the four additional models on the same suite are reported in Appendices B, C, D, and E. Results for LMSYS Chatbot Arena (Llama3.1-8B-Instruct) and the general ability benchmarks (Qwen3-4B) are reported in Appendices G and H, respectively.

³For the Goodfire Llama SAE, the choice of layer for SAE attachment was made by the creators of the SAE. We chose the Gemma Scope layer to be at a comparable depth. Generally, deeper layers have a tendency to represent higher-level, sentence- or discourse-level abstractions (Balcels et al., 2024; Shi et al., 2025).

4.2. Results

In this section, we report results for Llama3.1-8B-Instruct for the ten static benchmarks described above. We report analogous results for four additional models — Gemma2-2B-Instruct (Team et al., 2024), DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025), Mistral-7B-Instruct-v0.1 (Jiang et al., 2023), and Qwen3-4B (Yang et al., 2025) — in Appendices B, C, D, and E, demonstrating that CG generalizes across model families and SAE variants. We also demonstrate how CG can be used with arena-style benchmarks like LMSYS Chatbot Arena in Appendix G. To efficiently sift through the large number of results that can arise from our method (given the scale of the SAE concept dictionaries), we sometimes use another LLM (Gemini 2.5 Flash, in this case) to filter, group, and summarize sets of concept descriptions.

SAE Concept Relevance. As a complementary, more systematic lens, we also run an LLM-as-judge step that labels each concept as semantically relevant or irrelevant to each benchmark in the suite (see Appendix J for the prompt, per-benchmark percentages, and method). The relevant fraction varies widely across benchmarks: from 8.2% (WinoGrande) to 53.1% (Natural Questions) for the Gemma SAE dictionary (16K+ concepts), and from 4.9% to 34.5% for the Llama SAE dictionary (45K+ concepts). We release these per-concept relevance labels as a CSV alongside our code, and CG can be re-run on the relevance-filtered subset as an alternative analysis option.

4.2.1. BENCHMARK GAPS

Benchmarks Exhibit Skewed Representation Across Concepts. The cross-benchmark coverage distribution across all ten benchmarks is shown in Figure 3. The distribution exhibits a strong right skew, resulting in both over- and under-representation of concepts in our (somewhat typical) suite of benchmarks. This skew results in any standard mean-based summary statistics being dominated by the outliers on the right part of the tail — a small number of “top concepts” with high representation (high $X_{\text{bench}}^{(c)}$). Conversely, when a concept has consistently *low* coverage across all included benchmarks, it risks being systematically under-tested. Another undesirable pattern is substantial overlap across benchmarks in a single evaluation suite (Figure 28, Appendix F).

The top concepts mainly relate to starting new conversations and sports news, with a focus on football. Other prominent themes include syntax and attributes of articles. For example, the top 10 concepts by benchmark coverage included: **(56130)** “English Premier League football discussions, especially about Manchester teams”;

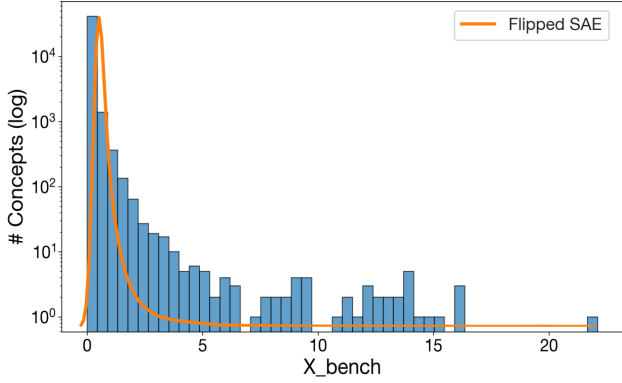


Figure 3. Cross-Benchmark Coverage. The distribution of $X_{\text{bench}}^{(c)}$ scores, indicating per-concept coverage across the 10 evaluated benchmarks, using the SAE of Llama 3.1 8B. This distribution exhibits strong right skew (most concepts have low coverage). Thus, average performance is dominated by a few concepts with high coverage (high $X_{\text{bench}}^{(c)}$). The orange curve shows a similar analysis using an SAE trained for a different model (Gemma 2 2B), resulting in qualitatively similar distributions (see Section 4.2.3).

(41290) “New conversation or topic segment boundary marker”.

Among the concepts with the *lowest* coverage score, one finds many concepts related to meta-cognition about the AI itself, e.g. its instructions, roleplaying boundaries, and how it discusses user inputs. The bottom 10 concepts by coverage include:

(53553) “The assistant should maintain professional boundaries when asked to roleplay”

(25352) “References to user messages or inputs in meta-discussion”.

We identified 314 concepts (1%) as entirely missing from this particular suite of benchmarks. These again include concepts related to the AI’s meta-cognition, as well as legal concepts. Examples include “The assistant explaining why it needs more information”, “The assistant needs to explain its limitations or capabilities”, and “Regulatory classification and compliance requirements”. Depending on the use case, these are plausibly important concepts to include in a benchmark suite, and these results demonstrate how CG can be used to identify such gaps.

Individual Benchmarks Miss Relevant Concepts. Individual benchmarks do show a similar skew in their representation (Appendix Figure 26), and every benchmark except Vectara misses at least 30% of all concepts (Appendix Figure 27). However, benchmarks are usually designed to evaluate a specific subset of capabilities, and so of course it may not be desirable for individual benchmarks to have

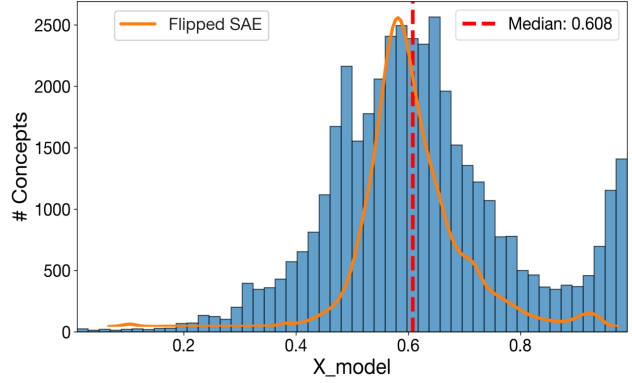


Figure 4. Cross-Benchmark Model Performance. The distribution of $X_{\text{model}}^{(c)}$ scores, indicating per-concept model performance for Llama 3.1 8B, evaluated across 10 benchmarks. The model exhibited high variance in performance across concepts. We observe particularly high performance for some concepts related to coding, data handling, instruction following, and expressing positive sentiment toward the user. The orange curve shows a similar analysis using an SAE trained for a different model (Gemma 2 2B), resulting in qualitatively similar distributions (see Section 4.2.3).

complete coverage of all concepts.

More importantly, we would like benchmarks to have coverage of *relevant* concepts. To identify such missing relevant concepts, we first used the CG method to identify all missing concepts for a given benchmark. Then, we used an LLM (Gemini 2.5 Flash, in this case) to identify missing concepts that one might expect to be in scope for the benchmark (see Appendix I.2 for the prompt). We also used the open-sourced web app to explore and verify these examples. This process exemplifies how one might use the CG method for unsupervised discovery of benchmark gaps.

Table 1 presents illustrative examples of concepts that are missed by benchmarks. These are concepts that seem central to the desired goals of the benchmarks, e.g. *AGI Eval* was missing “The need for thorough and objective assessment of evidence”, and *Social IQA* was missing “Instructions about how someone should behave or what qualities to embody”.

4.2.2. MODEL GAPS

In addition to benchmark gaps, we also analyze model performance gaps – on which SAE concepts do the models perform particularly well or particularly poor?

Model’s Best-Performing Concepts Include Coding and Commitment to Help. Because the SAE concepts are fixed for a given model, our method enables us to easily compare and create composite results across benchmarks. Figure 4 shows the distribution across concepts for

Table 1. Examples of Missing Relevant Concepts from Three Benchmarks.

Benchmark	Concept ID	Concept Description
AGI Eval	(33456)	The need for thorough and objective assessment of evidence
	(59559)	Careful qualification and nuanced explanation of complex topics
LogicBench	(56997)	The model is explaining how different elements or factors relate to each other
	(11957)	Mathematical and logical concepts across multiple languages
Social IQA	(35877)	Speaker defending or explaining their planned actions against expectations
	(1897)	Instructions about how someone should behave or what qualities to embody

cross-benchmark model performance, indicating that the model performs well on a number of concepts, with high performance across all benchmarks. Concepts with the highest cross-benchmark performance tended to concern STEM tasks (e.g., coding or data handling) or helpful behaviors (e.g., positive sentiments towards the user). The top 10 concepts included:

(20022) “Iteration or traversal through sequences in programming”,

(24074) “The assistant is about to provide an illustrative example”,

(2461) “Assistant expressing commitment to help or do its best”.

Model’s Worst-Performing Concepts Include Polite Rejection, Time, and Setting Boundaries. Perhaps more critical, from the standpoint of model evaluation, are the concepts which attained the poorest overall performance across benchmarks. An interesting recurring theme is that the worst-performing concepts include opposites to the helpful/sycophantic concepts discussed in the previous section (which came at the top in performance). Examples of worst-performing concepts include:

(26535) “The assistant needs to politely reject or redirect inappropriate requests”,

(56928) “The assistant maintaining professional boundaries while offering appropriate help”.

Furthermore, Competency Gaps identified other groups of competencies that have been anecdotally identified as LLM weaknesses in prior literature, validating our automated approach as a scalable and systematic method for identifying such model weaknesses. These include:

- **Representations of Time:**

(29324) “Historical date and time period formatting”

(12644) “Cooking time durations in recipe instructions”)

- **Image Manipulations:**

(30206) “Image contrast adjustments in photo editing and computer vision” ⁴

- **Palindromes / Reasoning over Letters:**

(56613) “Code examples and explanations of palindrome checking algorithms”

- **Mathematical Operations:**

(64527) “Mathematical addition operator in calculations”.

Additionally, the proposed method surfaces LLM weaknesses that have *not* been previously studied in the literature. One such category is “appeals to intuition in reasoning or decision making ”:

(64413) “The concept of grokking and deep intuitive understanding”

(64540) “Intuitive understanding and natural ease of use”

Moreover, the exploratory web application allows users to directly examine example data points and better understand how such competencies manifest in practice (see Figure 29).

4.2.3. ROBUSTNESS

Prior work has provided mixed evidence on the stability of SAE concepts (Paulo & Belrose, 2025; Menon et al., 2025). We therefore set out to evaluate the stability and generalizability of CG findings.

A Model-Specific SAE Is Not Necessary, and Different SAEs Can Yield Similar Results. We compared CG insights derived from Llama 3.1 8B using its own model-specific SAE, vs those obtained through Gemma 2 2B acti-

⁴Llama3.1-8B-Instruct is a text-only model, and we found that such concepts were activated by metadata indicating image editing, Photoshop scripts, and tutorials on the topic. This type of review of specific data points—both in the benchmarks and in the SAE training dataset—is enabled by our web app, and proves helpful for understanding the context and nuance of various concepts.

uations and SAE. As shown in Figures 3 and 4, the overall shape and medians of the score distributions were similar, especially considering the difference in dictionary sizes (Llama’s SAE contains 2.8x more concepts than Gemma’s). When comparing the best- and worst-performing concepts, we observed clear correspondences and similar interpretive insights, as shown in Table 4. This suggests that CG can yield meaningful insights even for LLMs *without* their own pre-trained SAE and demonstrates the overall stability of the method. Nonetheless, we expect that a model-specific SAE with a larger dictionary still offers the most precise and grounded results.

We emphasize that this cross-SAE robustness is *qualitative*, at the level of themes, rather than exact concept-by-concept identity. The two SAEs slice the same semantic regions at different granularities (e.g., Llama’s (2872) “Explaining time requirements and duration” aligns with Gemma’s broader (9936) “Dates and numeric sequences”), so Table 4 should be read as showing thematic — not bijective — correspondences. This is the level of consistency required for CG’s intended use as a tool for unsupervised gap discovery.

CG Scores Are Consistent Across Perturbations To assess the robustness of CG, we re-ran the full analysis 100 times, each time randomly dropping 20% of the examples per benchmark. The resulting standard deviations were low: 0.014 for X_{model} and 0.025 for X_{bench} on Llama. This suggests CG yields stable scores under random subsampling.

CG Scores Respond to Adversarial Perturbations. We conducted an adversarial ablation in which we identified the top 100 best- and worst-performing concepts, then removed the most salient 100 datapoints associated with them across all benchmarks. As expected, removing rows aligned with high-performing concepts lowered median X_{model} on average by 0.6%, while removing those aligned with low-performing concepts increased it on average by 1.3% (repeated across 10 repetitions). Despite removing less than 1% of the testing data, we were able to make predictable and consistent changes to the overall performance, suggesting that CG surfaces meaningful concept-level information.

5. Discussion

We have introduced a simple new method, Competency Gaps, which helps automatically identify gaps in benchmark coverage as well as gaps in model performance. We demonstrated the utility of this method, by performing example analyses on five models and a sampling of popular benchmarks. We also show that the method is robust to changes in the underlying sparse autoencoder and perturbations of the data.

Our benchmark gap analysis revealed a structural imbalance in a sample of popular benchmarks. For example, the benchmarks strongly emphasized concepts related to authority, control, and instruction-following, while neglecting complementary concepts related to polite refusals, meta-cognition, and meta-discussion about the AI itself. When evaluating across a benchmark suite, such skewed representation within the benchmarks may skew our perception of model capabilities. Our method also identified potential coverage gaps in specific benchmarks, pinpointing missing concepts that seemed relevant to each benchmark’s scope.

A similar bias toward sycophancy and instruction-following emerged in the model gap analysis. Here, positive or sycophantic concepts score highest, while opposing concepts (e.g. those linked to rejecting requests or setting boundaries) score lowest. While this is likely due in part to instruction-based post-training, we note that model gaps and benchmark gaps are heavily intertwined, and that benchmark gaps may lead to model gaps — model developers may unknowingly overlook (or be disincentivized to address) model weaknesses that are poorly covered by existing benchmarks. This entanglement is also visible in widely-used benchmarks: MMLU (Hendrycks et al., 2020) privileges factual recall over capabilities like calibrated refusal or meta-cognition, and SWE-Bench (Jimenez et al., 2024) encodes a narrow slice of software engineering (Python patches against unit tests). Models tuned for these benchmarks then look strongest along the concepts those benchmarks over-represent. Concretely, CG identifies *compilation* and *computer security* as among the most underrepresented concepts in SWE-Bench — consistent with the absence of any compiler or security repository among its twelve constituent projects — and surfaces missing historical coverage in MMLU (e.g., the Arab Spring and the Cuban Missile Crisis); we report these findings in Appendices H.1 and H.3.

5.1. Design Choices

Model-Internal SAE Concepts vs. Unified Text Embeddings. A natural alternative would be to use a model-independent text embedding to characterize benchmark items. We avoid this for two reasons: (1) external embeddings decouple the analysis from the model under evaluation, so surfaced “gaps” no longer reflect anything M itself represents; (2) such embeddings are typically dense and entangled, making the resulting axes harder to interpret than sparse SAE concepts. Using a model-internal SAE grounds the analysis in M , at the cost of being SAE-specific. For more on this trade-off, see Section 4.2.3.

Full-Dictionary vs. Filtered Reporting. Throughout the main results, we report statistics over the *full* SAE dictionary to avoid importing selection bias from a filtering criterion. The trade-off is that the full dictionary also includes

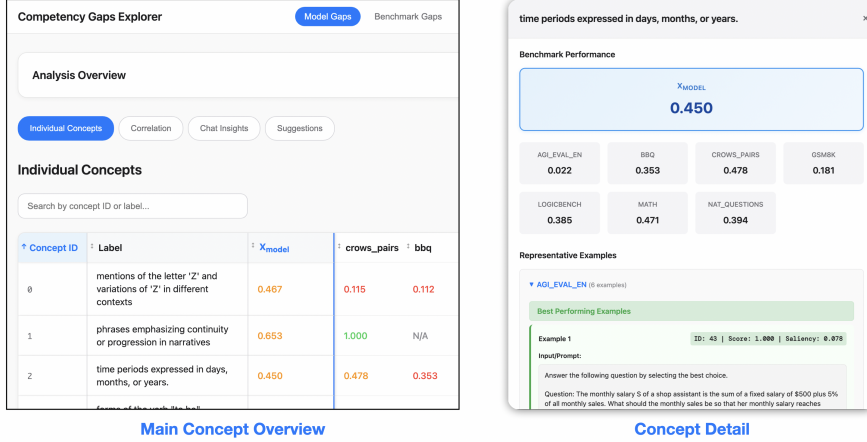


Figure 5. Exploratory Web Application. The CG method can output a large number of results, given the large size of most SAE concept dictionaries. This web interface enables easy, interactive exploration of CG results, including a searchable and filterable list of all concepts in the *Main Concept Overview*, with an expandable *Concept Detail Modal* that provides additional per-benchmark information, including specific example generations. The application also includes dedicated sections for cross-benchmark correlation, benchmark coverage inspection, and supplementary analyses. This web application is open-sourced. Additional screenshots in Appendix K.

low-quality or near-duplicate concepts that can make raw distributions noisier. Filtered analyses (e.g., LLM-selected critical concepts, as described in Appendix I.2) are useful as a post-hoc lens for human inspection, but should not be used for aggregate statistics.

5.2. Limitations

Concept Coverage is Limited to SAE Concept Space.

The usage of SAE concepts means that this method can only detect competency gaps for concepts that have *some* representation in the model; concepts that are absent from the model are by construction invisible to CG. Thus, we should take care in how we interpret our results: (1) *benchmark gap* analyses may reveal where models possess internal representations for concepts that evaluations fail to adequately test, and (2) *model gap* analyses may identify cases where models have flawed or partial internal representations that they cannot apply effectively to downstream tasks.

Concept Activation Is Not Capability. CG does not assume that activation of a concept implies competence in that concept. Capability is measured solely through the benchmark scoring function $m_b(i)$; concept activations are used only for attribution, weighting performance into the SAE space. Gaps are diagnosed jointly from activation and performance, not from either alone.

Uncontrolled Concept Difficulty. A concept’s CG score can be confounded by the difficulty of the items in which it activates: a low $X_{\text{model}}^{(c)}$ may reflect a genuine model gap or simply the fact that c activates on the hardest examples (the “hard-question confound”). Calibrating for item-level difficulty is an important direction for future work.

5.3. Downstream Applications and Future Work

Benchmark Search and Selection. Our method could be integrated into a database of available benchmarks (e.g., Hugging Face Datasets). Users seeking benchmarks to evaluate their models could use CG to inform their selection to achieve a desired coverage.

Targeted Creation of Novel Benchmark Data. Beyond characterizing existing benchmarks, a list of underrepresented concepts within the suite could guide the generation of novel benchmark data. Such data could be created either by prompting LLMs with autointerpretability labels or by directly applying SAE-based steering during generation.

Method Improvements. We hope that this work may serve as a starting point for further development on the method. For example, in future we may wish to incorporate an automated sensitivity analysis for the choice of SAE layer, or per-token activations for more fine-grained analyses.

Current model evaluations may risk a narrower view of “competence,” potentially leaving critical gaps untested. We hope that our method enables benchmark developers and model testers to identify both benchmark gaps and model gaps, uncovering and addressing weaknesses in areas that may be essential for real-world, human-facing use-cases.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning, and especially model evaluation. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

The authors would hereby like to thank the following colleagues, listed in alphabetical order, for helpful discussions: Tom Lieberum, Neel Nanda, Senthooran Rajamanoharan, and Jasper Snoek.

LLM Usage

LLMs were used in parts of the implementation and during the writing of the paper (e.g., paragraph shortening, transition refinement, etc.). AI-powered search engines were also used to help identify some references. LLM clustering was used to sift through large amounts of data produced by our method.

References

- Balcells, D., Lerner, B., Oesterle, M., Ucar, E., and Heimersheim, S. Evolution of SAE features across layers in LLMs. *arXiv preprint arXiv:2410.08869*, 2024.
- Balsam, D., McGrath, T., Gorton, L., Nguyen, N., Deng, M., and Ho, E. Announcing open-source SAEs for Llama 3.3 70B and Llama 3.1 8B, 2025. URL <https://www.goodfire.ai/blog/sae-open-source-announcement>.
- Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2, 2023.
- Burnell, R., Schellaert, W., Burden, J., Ullman, T. D., Martinez-Plumed, F., Tenenbaum, J. B., Rutar, D., Cheke, L. G., Sohl-Dickstein, J., Mitchell, M., et al. Rethink reporting of evaluation results in AI. *Science*, 380(6641): 136–138, 2023.
- Calian, D. A., Farquhar, G., Kemaev, I., Zintgraf, L. M., Hessel, M., Shar, J., Oh, J., György, A., Schaul, T., Dean, J., et al. DataRater: Meta-learned dataset curation. *arXiv preprint arXiv:2505.17895*, 2025.
- Chang, T. A. and Bergen, B. K. Language model behavior: A comprehensive survey. *Computational Linguistics*, 50 (1):293–350, 2024.
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., et al. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology*, 15(3):1–45, 2024.
- Chen, J. and Mueller, J. Automated data curation for robust language model fine-tuning. *arXiv preprint arXiv:2403.12776*, 2024.
- Cheng, J., Lu, Y., Gu, X., Ke, P., Liu, X., Dong, Y., Wang, H., Tang, J., and Huang, M. AutoDetect: Towards a unified framework for automated weakness detection in large language models. *arXiv preprint arXiv:2406.16714*, 2024.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Cosgrove, T. Mistral-7B sparse autoencoder (layer 16). <https://huggingface.co/tylercogrove/mistral-7b-sparse-autoencoder-layer16>, 2024.
- Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- Derczynski, L., Galinkin, E., Martin, J., Majumdar, S., and Inie, N. garak: A framework for security probing large language models. *arXiv preprint arXiv:2406.11036*, 2024.
- Didolkar, A., Goyal, A., Ke, N. R., Guo, S., Valko, M., Lillicrap, T., Jimenez Rezende, D., Bengio, Y., Mozer, M. C., and Arora, S. Metacognitive capabilities of LLMs: An exploration in mathematical problem solving. *Advances in Neural Information Processing Systems*, 37: 19783–19812, 2024.
- Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.
- Gan, E., Zhao, Y., Cheng, L., Mao, Y., Goyal, A., Kawaguchi, K., Kan, M.-Y., and Shieh, M. Reasoning robustness of LLMs to adversarial typographical errors. *arXiv preprint arXiv:2411.05345*, 2024.
- Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., and Wu, J. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D.,

- Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., Linde, J. v. d., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhota, K., Rantala-Yearly, L., Maaten, L. v. d., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., Oliveira, L. d., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vanden-hende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damla, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait,

- Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The Llama 3 herd of models, November 2024. URL <http://arxiv.org/abs/2407.21783>. arXiv:2407.21783.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Guo, Z., Jin, R., Liu, C., Huang, Y., Shi, D., Yu, L., Liu, Y., Li, J., Xiong, B., Xiong, D., et al. Evaluating large language models: A comprehensive survey. *arXiv preprint arXiv:2310.19736*, 2023.
- Gupta, V., Ross, C., Pantoja, D., Passonneau, R. J., Ung, M., and Williams, A. Improving model evaluation using smart filtering of benchmark datasets. *arXiv preprint arXiv:2410.20245*, 2024.
- Hanna, M. Qwen3-4B transcoders. <https://huggingface.co/mwhanna/qwen3-4b-transcoders>, 2024.
- Hardt, M. The emerging science of machine learning benchmarks. Online at <https://mlbenchmarks.org>, 2025. Manuscript.
- He, Z., Shu, W., Ge, X., Chen, L., Wang, J., Zhou, Y., Liu, F., Guo, Q., Huang, X., Wu, Z., et al. Llama Scope: Extracting millions of features from Llama-3.1-8B with sparse autoencoders. *arXiv preprint arXiv:2410.20526*, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.
- Hughes, S., Bae, M., and Li, M. Vectara hallucination leaderboard. <https://github.com/vectara/hallucination-leaderboard>, 2023.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Le Scao, T., Lavril, T., Wang, T., Lacroix, T., and El Sayed, W. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- Jimenez, C. E., Yang, J., Wetteg, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, volume 2024, pp. 54107–54157, 2024.
- Jones, E. and Steinhardt, J. Capturing failures of large language models via human cognitive biases. *Advances in Neural Information Processing Systems*, 35:11785–11799, 2022.
- Kahng, M., Tenney, I., Pushkarna, M., Liu, M. X., Wexler, J., Reif, E., Kallarakal, K., Chang, M., Terry, M., and Dixon, L. LLM comparator: Visual analytics for side-by-side evaluation of large language models. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pp. 1–7, 2024.
- Kim, E., Garg, A., Peng, K., and Garg, N. Correlated errors in large language models. *arXiv preprint arXiv:2506.07962*, 2025.
- Kim, S., Shin, J., Cho, Y., Jang, J., Longpre, S., Lee, H., Yun, S., Shin, S., Kim, S., Thorne, J., et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- Koo, R., Lee, M., Raheja, V., Park, J. I., Kim, Z. M., and Kang, D. Benchmarking cognitive biases in large language models as evaluators. *arXiv preprint arXiv:2309.17012*, 2023.
- Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Li, T., Chiang, W.-L., Frick, E., Dunlap, L., Wu, T., Zhu, B., Gonzalez, J. E., and Stoica, I. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*, 2024.
- Lieberum, T., Rajamanoharan, S., Conmy, A., Smith, L., Sonnerat, N., Varma, V., Kramár, J., Dragan, A., Shah, R., and Nanda, N. Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2, August 2024. URL <http://arxiv.org/abs/2408.05147>. arXiv:2408.05147.
- Manerba, M. M., Stańczak, K., Guidotti, R., and Augenstein, I. Social bias probing: Fairness benchmarking for language models. *arXiv preprint arXiv:2311.09090*, 2023.
- Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., et al. Harm-Bench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.
- McGrath, T., Balsam, D., Deng, M., and Ho, E. Understanding and steering Llama 3 with sparse autoencoders. *Goodfire Research*, September 2024.

- Menon, A., Shrivastava, M., Krueger, D., and Lubana, E. S. Analyzing (in)abilities of SAEs via formal languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4837–4862, 2025.
- Merrill, M. A., Shaw, A. G., Carlini, N., Li, B., Raj, H., Bercovich, I., Shi, L., Shin, J. Y., Walshe, T., Buchanan, E. K., et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Miller, J. K. and Tang, W. Evaluating LLM metrics through real-world capabilities. *arXiv preprint arXiv:2505.08253*, 2025.
- Mizrahi, M., Kaplan, G., Malkin, D., Dror, R., Shahaf, D., and Stanovsky, G. State of what art? a call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024.
- Nada, H., Asma, G., Ryan, M., Emily, R., Jimbo, W., Nithum, T., and Lucas, D. Can large language models explain their internal mechanisms? Explorable, Google PAIR, jul 2024. <https://pair.withgoogle.com/explorables/patchscopes/>.
- Nangia, N., Vania, C., Bhalerao, R., and Bowman, S. R. CrowS-Pairs: A challenge dataset for measuring social biases in masked language models. *arXiv preprint arXiv:2010.00133*, 2020.
- Parmar, M., Patel, N., Varshney, N., Nakamura, M., Luo, M., Mashetty, S., Mitra, A., and Baral, C. Towards systematic evaluation of logical reasoning ability of large language models. *arXiv preprint arXiv:2404.15522*, 2024.
- Parrish, A., Chen, A., Nangia, N., Padmakumar, V., Phang, J., Thompson, J., Htut, P. M., and Bowman, S. R. BBQ: A hand-built bias benchmark for question answering. *arXiv preprint arXiv:2110.08193*, 2021.
- Paulo, G. and Belrose, N. Sparse autoencoders trained on the same data learn different features. *arXiv preprint arXiv:2501.16615*, 2025.
- Perez, E., Ringer, S., Lukosiute, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., et al. Discovering language model behaviors with model-written evaluations. In *Findings of the association for computational linguistics: ACL 2023*, pp. 13387–13434, 2023.
- Phan, L., Gatti, A., Han, Z., Li, N., Hu, J., Zhang, H., Zhang, C. B. C., Shaaban, M., Ling, J., Shi, S., et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Sap, M., Rashkin, H., Chen, D., LeBras, R., and Choi, Y. SocialQA: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.
- Shi, W., Li, S., Liang, T., Wan, M., Ma, G., Wang, X., and He, X. Route sparse autoencoder to interpret large language models. *arXiv preprint arXiv:2503.08200*, 2025.
- Shu, D., Wu, X., Zhao, H., Rai, D., Yao, Z., Liu, N., and Du, M. A survey on sparse autoencoders: Interpreting the internal mechanisms of large language models. *arXiv preprint arXiv:2503.05613*, 2025.
- Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J.-B., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozńska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshv, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucińska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., Amersfoort, J. v., Gordon, J., Lipschultz, J., Newlan, J., Ji, J.-y., Mohamed, K., Badola, K., Black, K., Millican, K., McDonnell, K., Nguyen, K., Sodhia, K., Greene, K., Sjoesund, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., McNealus, L., Soares, L. B., Kilpatrick, L., Dixon, L., Martins, L., Reid, M., Singh, M., Iverson, M., Görner, M., Velloso, M., Wirth, M., Davidow, M., Miller, M., Rahtz, M., Watson, M., Risdal, M., Kazemi, M., Moynihan, M., Zhang, M., Kahng, M., Park, M., Rahman, M., Khatwani, M., Dao, N., Bardoliwalla, N., Devanathan, N., Dumai, N., Chauhan, N., Wahltinez, O., Botarda, P., Barnes, P., Barham, P., Michel, P., Jin, P., Georgiev, P., Culliton, P., Kuppala, P., Comanescu, R., Merhej, R., Jana, R., Rokni, R. A., Agarwal, R., Mullins, R., Saadat, S., Carthy, S. M., Cogan, S., Perrin, S., Arnold, S. M. R.,

- Krause, S., Dai, S., Garg, S., Sheth, S., Ronstrom, S., Chan, S., Jordan, T., Yu, T., Eccles, T., Hennigan, T., Kocisky, T., Doshi, T., Jain, V., Yadav, V., Meshram, V., Dharmadhikari, V., Barkley, W., Wei, W., Ye, W., Han, W., Kwon, W., Xu, X., Shen, Z., Gong, Z., Wei, Z., Cotruta, V., Kirk, P., Rao, A., Giang, M., Peran, L., Warkentin, T., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Sculley, D., Banks, J., Dragan, A., Petrov, S., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Borgeaud, S., Fiedel, N., Joulin, A., Kenealy, K., Dadashi, R., and Andreev, A. Gemma 2: Improving open language models at a practical size, October 2024. URL <http://arxiv.org/abs/2408.00118>. arXiv:2408.00118.
- Templeton, A. *Scaling monosemanticity: Extracting interpretable features from Claude 3 sonnet*. Anthropic, 2024.
- Tjuaatja, L. and Neubig, G. BehaviorBox: Automated discovery of fine-grained performance differences between language models. *arXiv preprint arXiv:2506.02204*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, Z., Meng, Z., Zheng, X., and Wattenhofer, R. Assessing adversarial robustness of large language models: An empirical study. *arXiv preprint arXiv:2405.02764*, 2024.
- Yin, J., Bose, A., Cong, G., Lyngaas, I., and Anthony, Q. Comparative study of large language model architectures on frontier. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 556–569. IEEE, 2024.
- Yin, Z., Wang, Z., Palikhe, A., and Zhang, W. Fairness definitions in language models explained. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 16(1):e70063, 2026.
- Zeng, Z., Wang, Y., Hajishirzi, H., and Koh, P. W. EvalTree: Profiling language model weaknesses via hierarchical capability trees. *arXiv preprint arXiv:2503.08893*, 2025.
- Zheng, C., Beltran-Velez, N., Karlekar, S., Shi, C., Nazaret, A., Mallik, A., Feder, A., and Blei, D. M. Model directions, not words: Mechanistic topic models using sparse autoencoders. *arXiv preprint arXiv:2507.23220*, 2025.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623, 2023.
- Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., and Duan, N. AGIEval: A human-centric benchmark for evaluating foundation models, 2023.
- Zhong, Z. and Raghunathan, A. Watch the weights: Unsupervised monitoring and control of fine-tuned LLMs. *arXiv preprint arXiv:2508.00161*, 2025.

Appendix

Table of Contents

A	Sparse Autoencoder (SAE) Primer	17
B	Additional Results: Gemma 2 2B Instruct	18
B.1	Benchmark Gaps	18
B.2	Model Gaps	20
B.3	Robustness	20
C	Additional Results: DeepSeek-R1-Distill-Llama-8B	22
C.1	Benchmark Gaps	22
C.2	Model Gaps	24
D	Additional Results: Mistral-7B-Instruct-v0.1	26
D.1	Benchmark Gaps	26
D.2	Model Gaps	27
E	Additional Results: Qwen3-4B	29
E.1	Benchmark Gaps	29
E.2	Model Gaps	30
E.3	Causal Validation: Foreign-Name Bias	30
F	Additional Results: Llama 3.1 8B Instruct	32
F.1	Benchmark Gaps	32
F.2	Model Gaps	34
G	Additional Results: LMSYS Chatbot Arena	35
G.1	Implementation Details	35
G.2	Benchmark Gaps	35
G.3	Model Gaps	36
H	Additional Results: Conventional Benchmarks	37
H.1	SWE-Bench	37
H.2	Terminal-Bench	38
H.3	MMLU	38
H.4	GPQA	40
H.5	Humanity’s Last Exam (HLE)	41
I	Prompts for LLM Analysis of CG Results	42
I.1	Benchmark Gaps: Missing Concepts (Cross-Benchmark)	42
I.2	Benchmark Gaps: Missing Concepts (Per-Benchmark)	42
I.3	Benchmark Gaps: Matching	42
J	LLM-as-Judge Per-Benchmark Concept Relevance	43
K	Exploratory Web Application: Additional Screenshots	45
L	Comparison with Other Methods	53

L.1	Comparison Methodology	53
L.2	Benchmark Gaps Overview	53
L.3	Model Gaps Overview	54
M	Comparison with Other Methods: garak	55
M.1	Llama 3.1 8B Instruct	56
M.2	Gemma 2 2B Instruct	58
N	Comparison with Other Methods: AutoDetect	60
N.1	Llama 3.1 8B Instruct	61
N.2	Gemma 2 2B Instruct	63
O	Comparison with Other Methods: Arena-Hard-Auto	65
O.1	AGI Eval EN	66
O.2	BBQ	66
O.3	CROWS Pairs	67
O.4	GSM8K	68
O.5	LogicBench	69
O.6	MATH	69
O.7	Natural Questions	70
O.8	Real Toxicity	70
O.9	Social IQA	71
O.10	Vectara	72
O.11	WinoGrande	73
P	Comparison with Other Methods: EvalTree	74
P.1	MATH	74

A. Sparse Autoencoder (SAE) Primer

We summarize the SAE terminology used throughout the paper. A sparse autoencoder is trained to reconstruct the internal activations of a host model M at a chosen layer, using an over-complete dictionary of K features with an explicit sparsity penalty (Bricken et al., 2023; Cunningham et al., 2023). The goal is to recover monosemantic “concepts” that decompose superposed neuron activations into interpretable axes (Elhage et al., 2022).

We refer to each of the K learned feature directions as a *concept*, the full set as the *concept dictionary* C_{SAE} , and the space they span as the *SAE space*. The activation of concept c on token $x_{i,j}$ is denoted $SAE(x_{i,j}, c) \in \mathbb{R}_{\geq 0}$. For each concept, an autointerpretability pipeline assigns a textual label l_c by prompting an LLM with high-activation exemplars (McGrath et al., 2024).

The dictionary size K is not a hyperparameter we tune; it is fixed by the pre-trained SAE’s *expansion factor* over the host model’s hidden dimension, with prior scaling work suggesting K should grow with model size (Templeton, 2024; Gao et al., 2024). The Goodfire SAE for Llama 3.1 8B uses $K \approx 65,536$ concepts; the Gemma Scope SAE for Gemma 2 2B uses $K \approx 16,384$. Larger dictionaries yield finer-grained concepts but leave our analysis pipeline unchanged.

Running an SAE at a single layer adds one linear projection of size $d_{\text{model}} \times K$ per token plus a sparsity step. The host LLM, by comparison, performs many such projections per layer across all of its layers, plus attention — so the SAE contributes roughly one extra “layer’s worth” of compute on top of the full forward pass, typically a few percent for an 8B model at a single residual-stream layer.

B. Additional Results: Gemma 2 2B Instruct

B.1. Benchmark Gaps

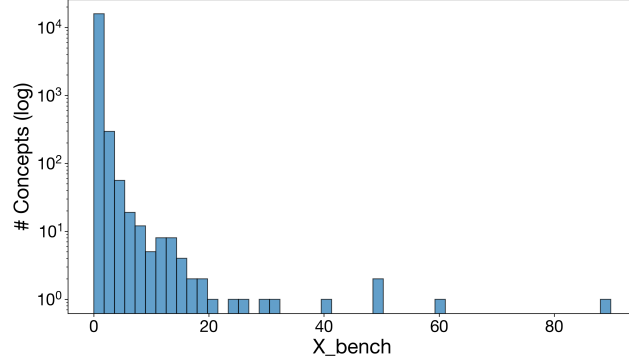


Figure 6. Cross-Benchmark Coverage. The distribution of $X_{\text{bench}}^{(c)}$ scores obtained for the 10 evaluated benchmarks, using the SAE of Gemma 2 2B.

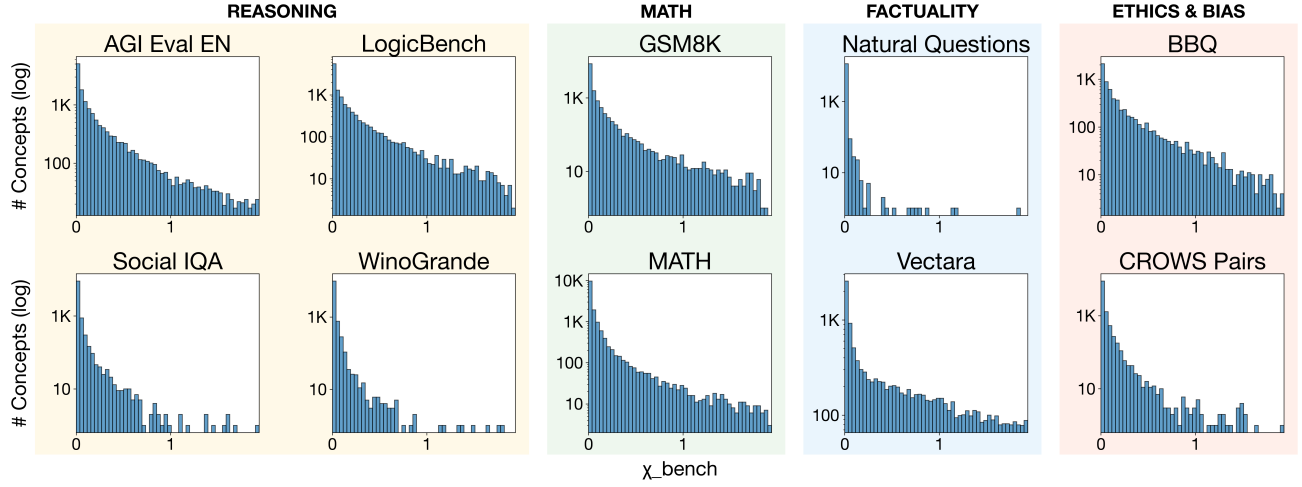


Figure 7. Coverage Within Individual Benchmarks. A breakdown of $X_{\text{bench}}^{(b,c)}$ score distributions for individual benchmarks obtained via Gemma 2 2B.

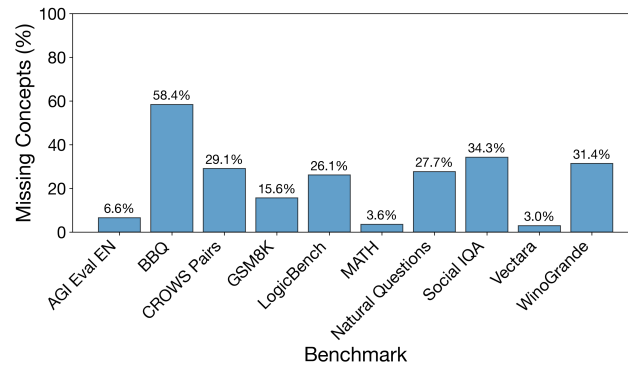


Figure 8. Missing Concepts. Proportion of the SAE concept dictionary for Gemma 2 2B that is not tested by the respective benchmarks.

Table 2. Examples of Missing Concepts from the Full Benchmark Suite.

Concept ID	Concept Description
(5169)	patterns of repeated characters or symbols
(3674)	phrases related to coding or programming syntax
(9514)	detailed references and citations in academic writing
(10108)	questions and references to uncertainty or confusion
(5102)	specific programming or technical terminology related to data storage and handling

Table 3. Examples of Missing Concepts from Individual Benchmarks.

Benchmark	Concept ID	Concept Description
AGI Eval	(12792)	phrases related to problem-solving or troubleshooting
	(12436)	error handling and debugging statements in programming code
LogicBench	(7873)	phrases that indicate conditions or states related to certainty or necessity
	(7264)	occurrences of mathematical or formal logic terms and control structures in the text
Social IQA	(2863)	concepts related to social dynamics and collaborative efforts
	(12897)	concepts related to socio-cultural analysis and individualized experiences

B.2. Model Gaps

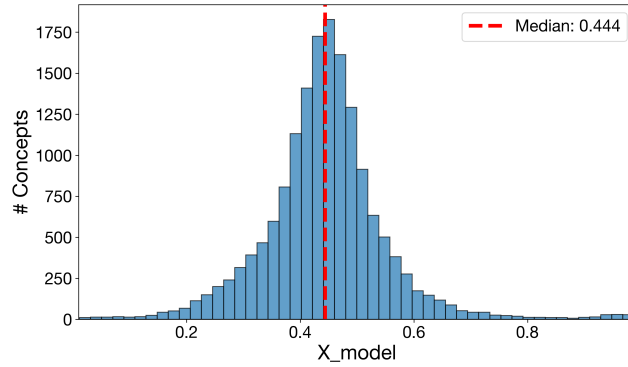


Figure 9. Cross-Benchmark Performance. The distribution of $X_{\text{model}}^{(c)}$ scores obtained for for Gemma 2 2B.

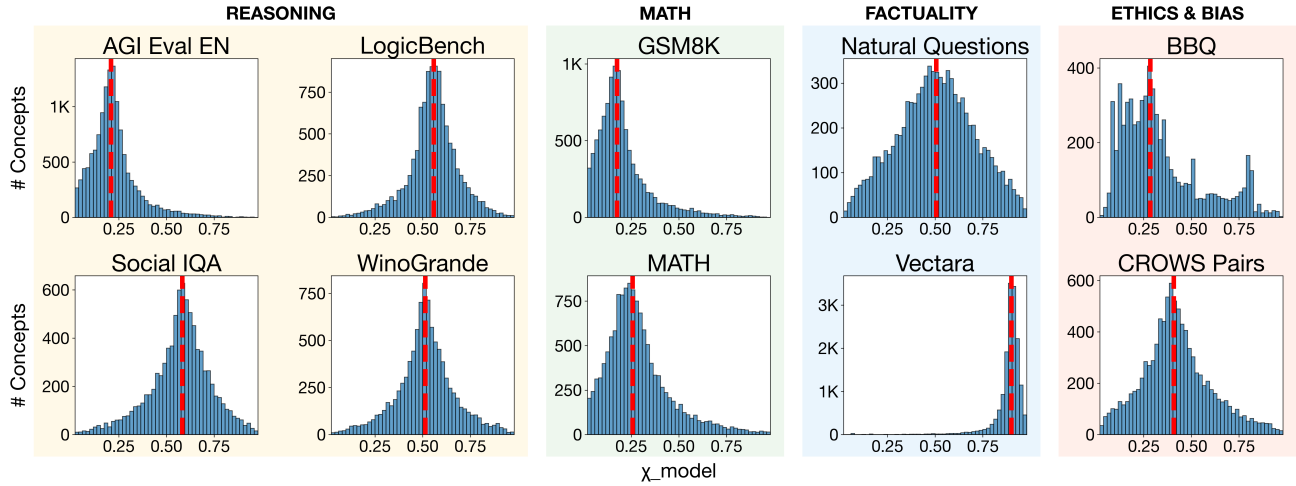


Figure 10. Per-Benchmark Distributions for Model Performance. A breakdown of model performance $\chi_{\text{model}}^{(b,c)}$ score distributions for individual benchmarks obtained for Gemma 2 2B. The red line indicates the median.

B.3. Robustness

Perturbation Consistency. We re-ran the full analysis 100 times, each time randomly dropping 20% of the examples per benchmark. The resulting standard deviations were: 0.012 for X_{model} and 0.011 for X_{bench} .

Adversarial Perturbations. Upon removal of the most salient 100 datapoints associated with top 100 best-performing concepts across all benchmarks lowered median X_{model} on average by 0.8%. On the other hand, removing the most salient 100 datapoints associated with top 100 worst-performing concepts across all benchmarks increased median X_{model} on average by 0.5%. This process was repeated 10 times.

Table 4. Examples of Concept Correspondences in Best/Worst Performance and Coverage Analyses. The analysis was conducted for Llama 3.1 8B: the Llama SAE results used the model’s own activation and custom SAE; the Gemma SAE results used Gemma 2 2B’s activations and SAE. They demonstrate reasonable correspondence when the SAEs are swapped.

Analysis	Llama SAE	Gemma SAE
<i>Best Performance</i>	(45314) Legal reasoning and argumentation patterns in multiple choice questions (27510) Code patterns for including JavaScript resources in web pages	(5471) References to legal cases and procedural aspects of law (8196) Code constructs or reserved keywords in programming languages
<i>Worst Performance</i>	(2874) Mathematical differentiation operators and notation (2872) Explaining time requirements and duration	(13908) Numerical values, counts or measurements (9936) Dates and numeric sequences
<i>Best Coverage</i>	(41290) New conversation or topic segment boundary marker (902) Step-by-step mathematical explanations and calculations	(11527) The start of a document (11880) Mathematical expressions and calculations related to derivatives and factors
<i>Worst Coverage</i>	(27900) Discussions of factual accuracy and consistency checking (47946) The assistant explains how it processes and handles information	(5657) Terms related to correctness and accuracy in responses or answers (1797) Phrases related to instructions or operational processes

C. Additional Results: DeepSeek-R1-Distill-Llama-8B

We apply CG to DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025) using the Llama-Scope-R1-Distill SAE (residual stream, layer 20) (He et al., 2024).

C.1. Benchmark Gaps

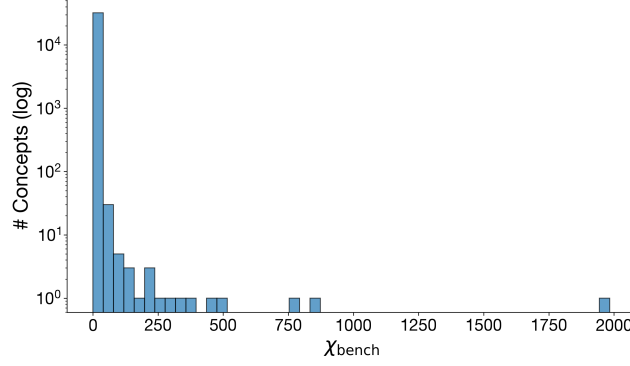


Figure 11. Cross-Benchmark Coverage. The distribution of $X_{\text{bench}}^{(c)}$ scores obtained for the 10 evaluated benchmarks, using the SAE of DeepSeek-R1-Distill-Llama-8B.

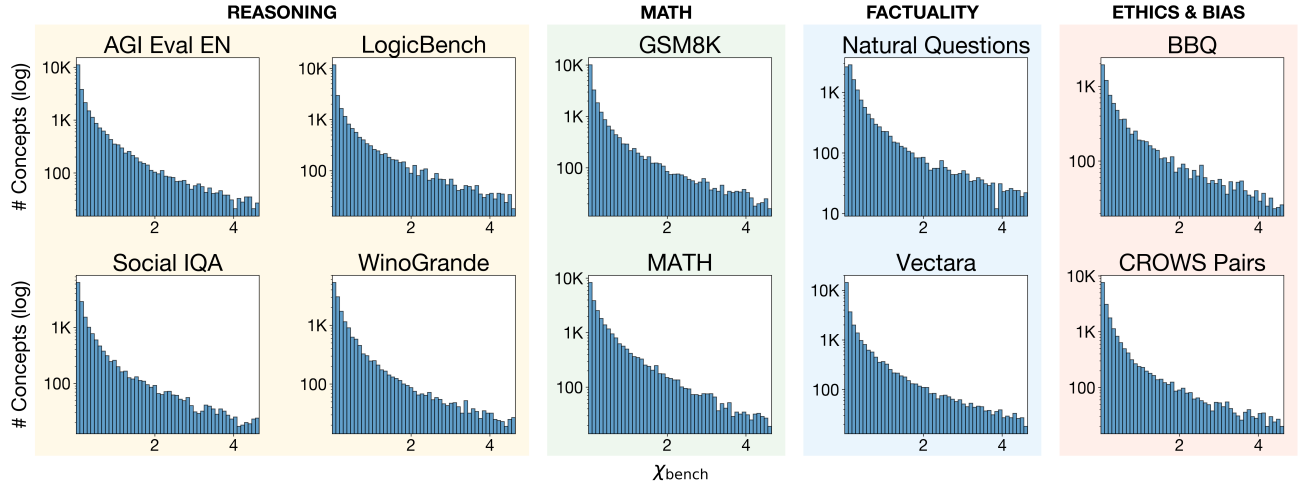


Figure 12. Coverage Within Individual Benchmarks. A breakdown of $X_{\text{bench}}^{(b,c)}$ score distributions for individual benchmarks obtained via DeepSeek-R1-Distill-Llama-8B.

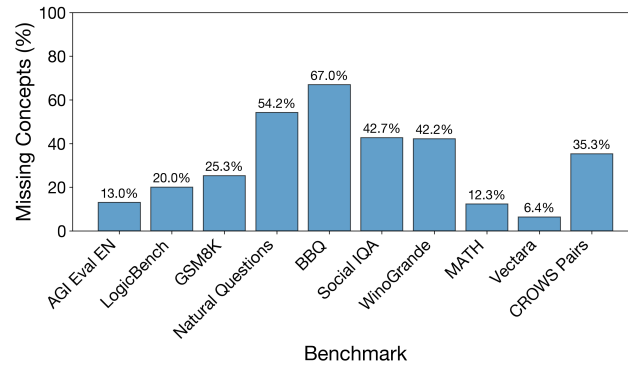


Figure 13. Missing Concepts. Proportion of the SAE concept dictionary for DeepSeek-R1-Distill-Llama-8B that is not tested by the respective benchmarks.

Table 5. Examples of Missing Concepts from the Full Benchmark Suite (DeepSeek-R1-Distill-Llama-8B).

Concept ID	Concept Description
(4492)	mathematical reasoning and trigonometry
(5054)	text that contains mathematical reasoning including variables, angles, equations, and calculations
(3936)	the word “opinion” and its derivatives
(12843)	code snippets relating to requests, tags, and parameters
(799)	words ending in “ic, ics, ist, ied, al” or their inflections

Table 6. Examples of Missing Concepts from Individual Benchmarks (DeepSeek-R1-Distill-Llama-8B).

Benchmark	Concept ID	Concept Description
AGI Eval	(699)	mentions of museums or libraries, or potentially things you might find in those places
	(30886)	information about Arabic media sources
LogicBench	(24511)	words related to math problems and solutions
	(8523)	the word “multiple” or “multiples” appearing in mathematical contexts
Social IQA	(8971)	code comments and code fragments
	(17303)	the words “root” or “roots”, as well as legal terms

C.2. Model Gaps

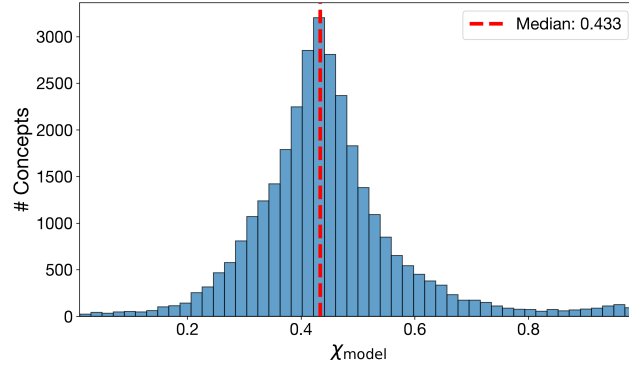


Figure 14. Cross-Benchmark Performance. The distribution of $X_{\text{model}}^{(c)}$ scores obtained for DeepSeek-R1-Distill-Llama-8B.

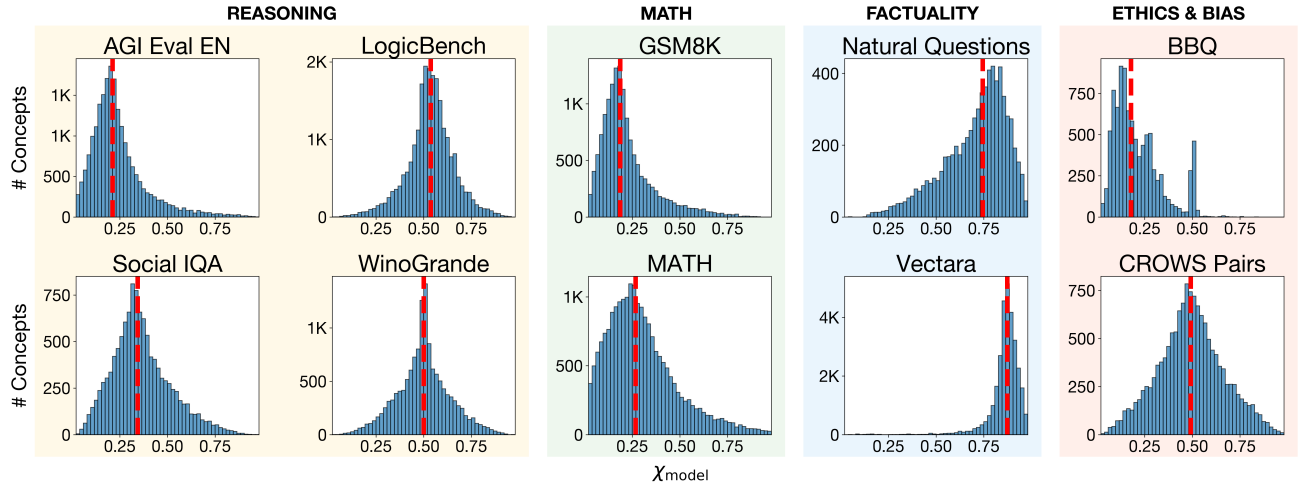


Figure 15. Per-Benchmark Distributions for Model Performance. A breakdown of model performance $X_{\text{model}}^{(b,c)}$ score distributions for individual benchmarks obtained for DeepSeek-R1-Distill-Llama-8B. The red line indicates the median.

Table 7. Examples of Best- and Worst-Performing Concepts (DeepSeek-R1-Distill-Llama-8B). Filtered to concepts that activate in at least 5 benchmarks with ≥ 50 supporting datapoints. The pattern is suggestive: R1-Distill scores well on the surface forms of math reasoning (variables, LaTeX, reasoning-step transitions) but poorly on its substance (multi-digit arithmetic, boxed numerical answers, code syntax).

	Concept ID	Concept Description
<i>Best Performance</i>	(14585)	mentions of variables, specifically within mathematical or logical reasoning
	(17004)	LaTeX math code
	(17718)	the word “Alternatively” which is used to introduce different reasoning steps to solve math problems
<i>Worst Performance</i>	(32637)	boxed numerical answers in a mathematical context
	(24736)	multi-digit numbers
	(9878)	code blocks in C# with curly braces

D. Additional Results: Mistral-7B-Instruct-v0.1

We apply CG to Mistral-7B-Instruct-v0.1 (Jiang et al., 2023) using a publicly available SAE attached at layer 16 (MLP output) (Cosgrove, 2024). The autointerpretability labels for this SAE are short, terse fragments rather than full descriptions, so the example tables below should be read as illustrative of *which* concepts CG surfaces rather than as fully interpretable concept names.

D.1. Benchmark Gaps

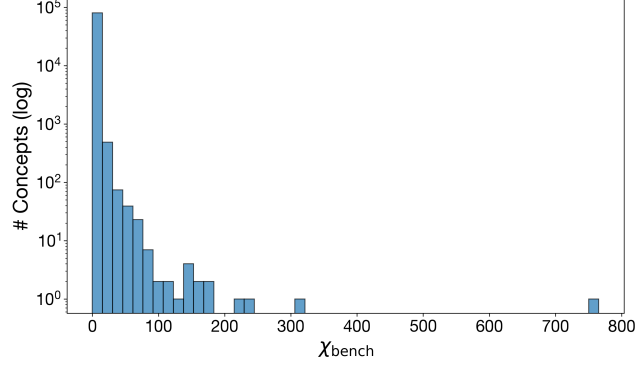


Figure 16. Cross-Benchmark Coverage. The distribution of $X_{\text{bench}}^{(c)}$ scores obtained for the 10 evaluated benchmarks, using the SAE of Mistral-7B-Instruct-v0.1.

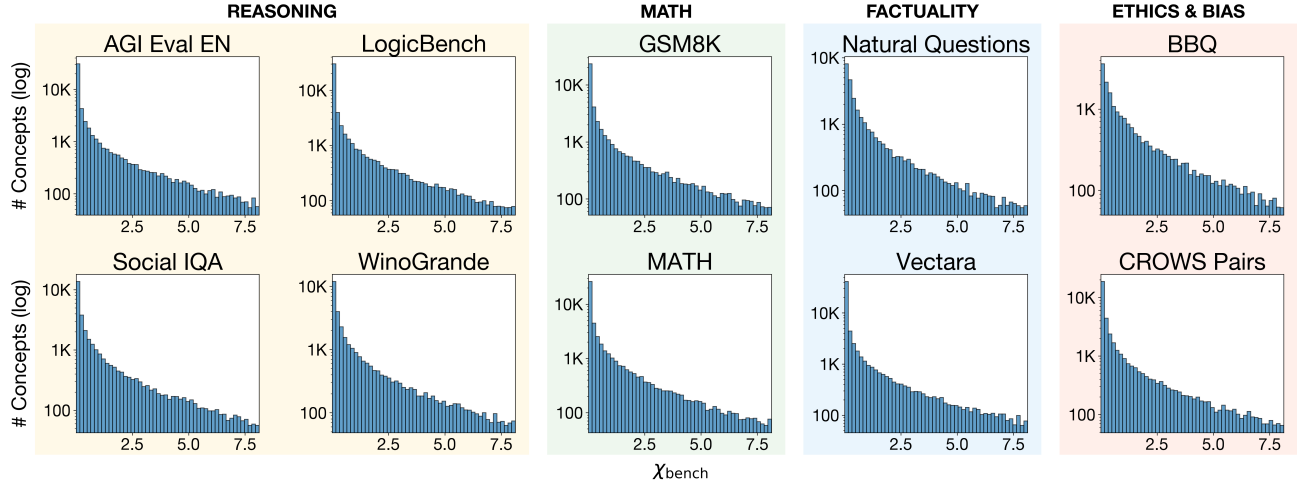


Figure 17. Coverage Within Individual Benchmarks. A breakdown of $X_{\text{bench}}^{(b,c)}$ score distributions for individual benchmarks obtained via Mistral-7B-Instruct-v0.1.

Uncovering Competency Gaps

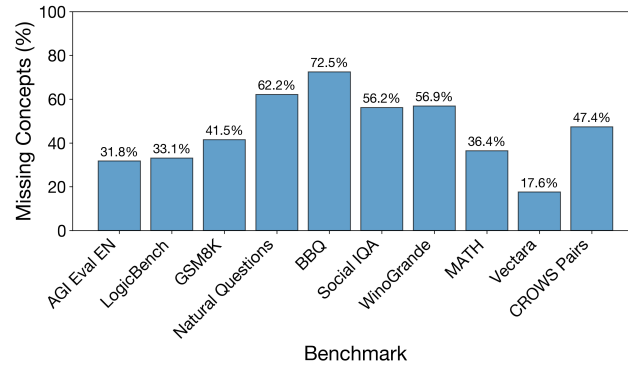


Figure 18. Missing Concepts. Proportion of the SAE concept dictionary for Mistral-7B-Instruct-v0.1 that is not tested by the respective benchmarks.

D.2. Model Gaps

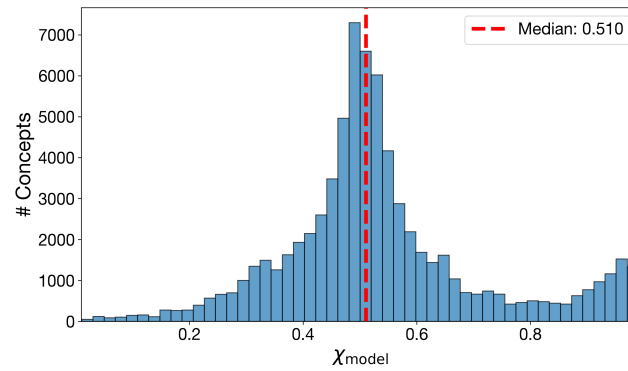


Figure 19. Cross-Benchmark Performance. The distribution of $X_{\text{model}}^{(c)}$ scores obtained for Mistral-7B-Instruct-v0.1.

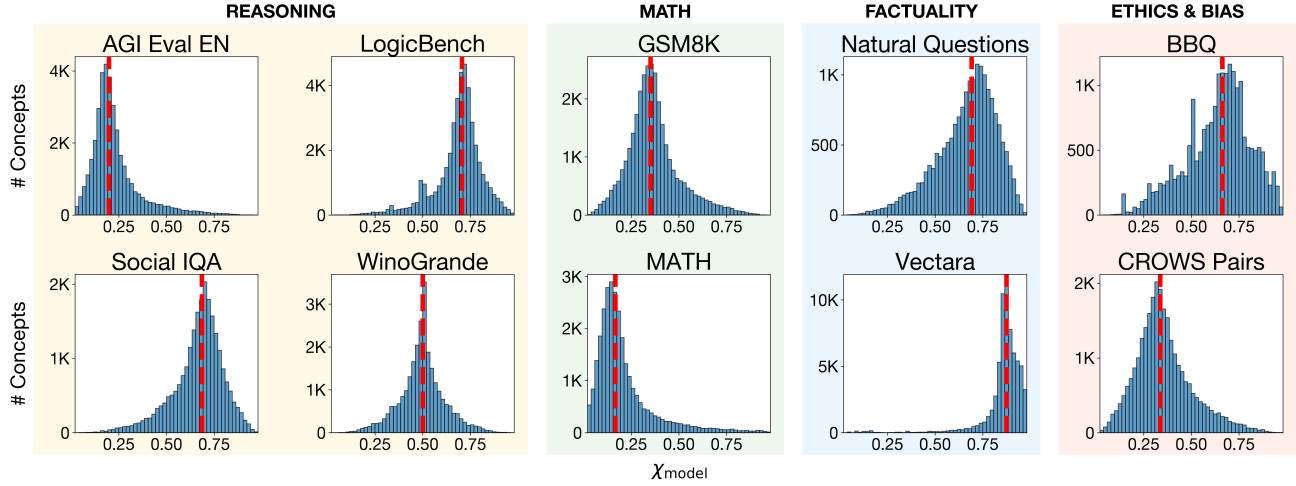


Figure 20. Per-Benchmark Distributions for Model Performance. A breakdown of model performance $\chi_{\text{model}}^{(b,c)}$ score distributions for individual benchmarks obtained for Mistral-7B-Instruct-v0.1. The red line indicates the median.

Table 8. Examples of Best- and Worst-Performing Concepts (Mistral-7B-Instruct-v0.1). Filtered to concepts that activate in at least 5 benchmarks with ≥ 50 supporting datapoints, and further restricted to entries with coherent semantic labels. Coherent labels are sparse for this SAE, so the table is shorter than for the other models.

	Concept ID	Concept Description
<i>Best Performance</i>	(14155)	Months of the year
	(101008)	automaticity, self-acting
	(64910)	drawing attributes
<i>Worst Performance</i>	(18081)	Invention and innovation
	(97261)	vehicles and assembly

E. Additional Results: Qwen3-4B

We apply CG to Qwen3-4B (Yang et al., 2025) using a publicly available set of transcoders on its residual stream (Hanna, 2024). Autointerpretability labels for these transcoders are available for a subset of concepts and tend to be short fragments; the example tables below restrict to that subset and to concepts with broad cross-benchmark support.

E.1. Benchmark Gaps

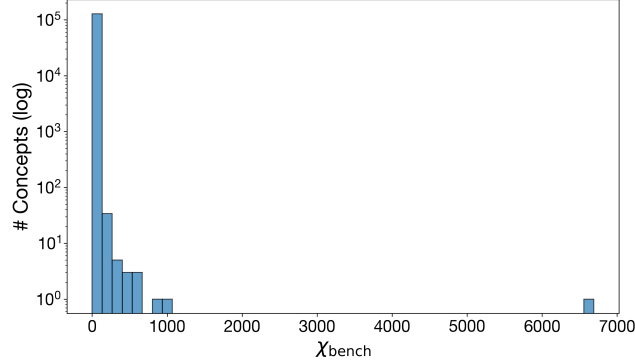


Figure 21. Cross-Benchmark Coverage. The distribution of $X_{\text{bench}}^{(c)}$ scores obtained for the 10 evaluated benchmarks, using the SAE of Qwen3-4B.

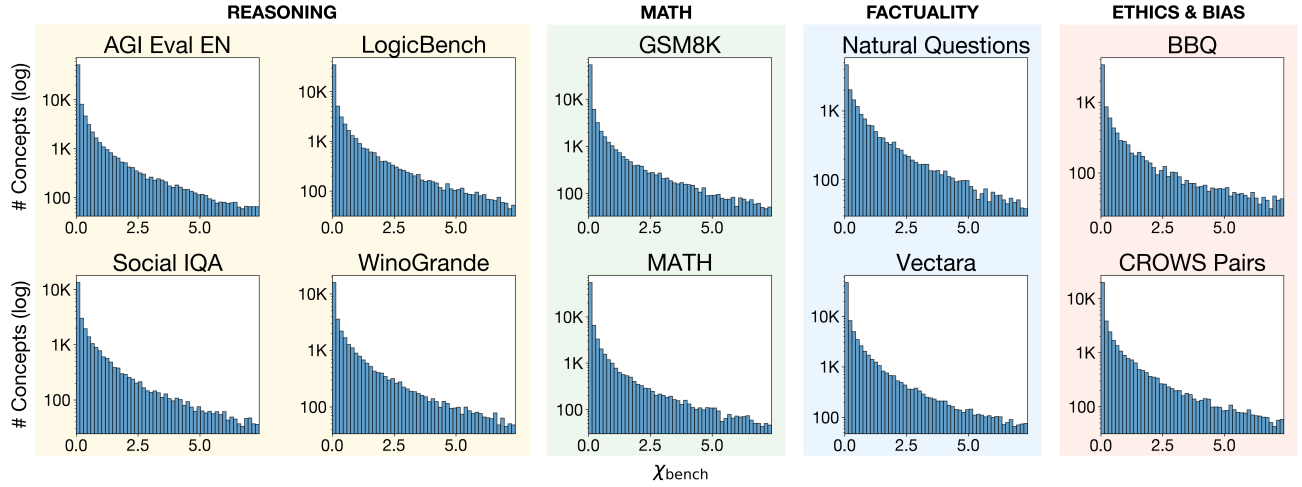


Figure 22. Coverage Within Individual Benchmarks. A breakdown of $X_{\text{bench}}^{(b,c)}$ score distributions for individual benchmarks obtained via Qwen3-4B.

Uncovering Competency Gaps

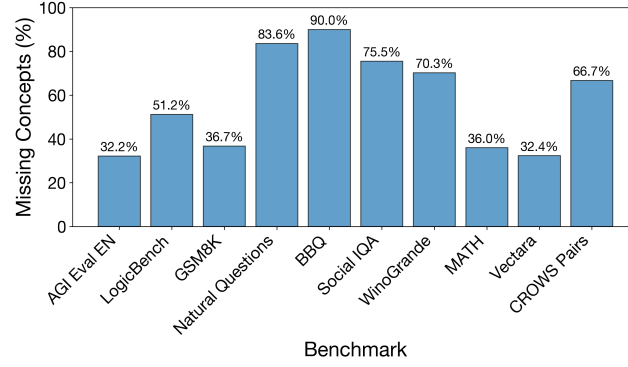


Figure 23. Missing Concepts. Proportion of the SAE concept dictionary for Qwen3-4B that is not tested by the respective benchmarks.

E.2. Model Gaps

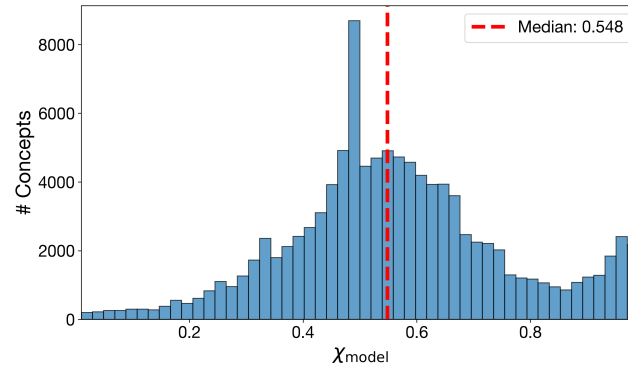


Figure 24. Cross-Benchmark Performance. The distribution of $X_{\text{model}}^{(c)}$ scores obtained for Qwen3-4B.

E.3. Causal Validation: Foreign-Name Bias

Beyond surfacing concepts associated with poor performance, CG can also guide targeted experiments to causally validate the identified model gaps. As an illustration, we examined concept **(19086)** “Foreign languages, names”, which received a low $X_{\text{model}}^{(c)}$ score for Qwen3-4B. We constructed a LogicBench-style prompt with two protagonists and ran it 10 times in each of two settings: (A) protagonists unnamed, and (B) protagonists named “Omar” and “Mahmoud”. The prompts were otherwise identical. Qwen3-4B answered correctly 80% of the time in setting (A) but only 30% of the time in setting (B), providing direct evidence that the foreign-name model gap surfaced by CG corresponds to a real, behaviorally observable weakness.

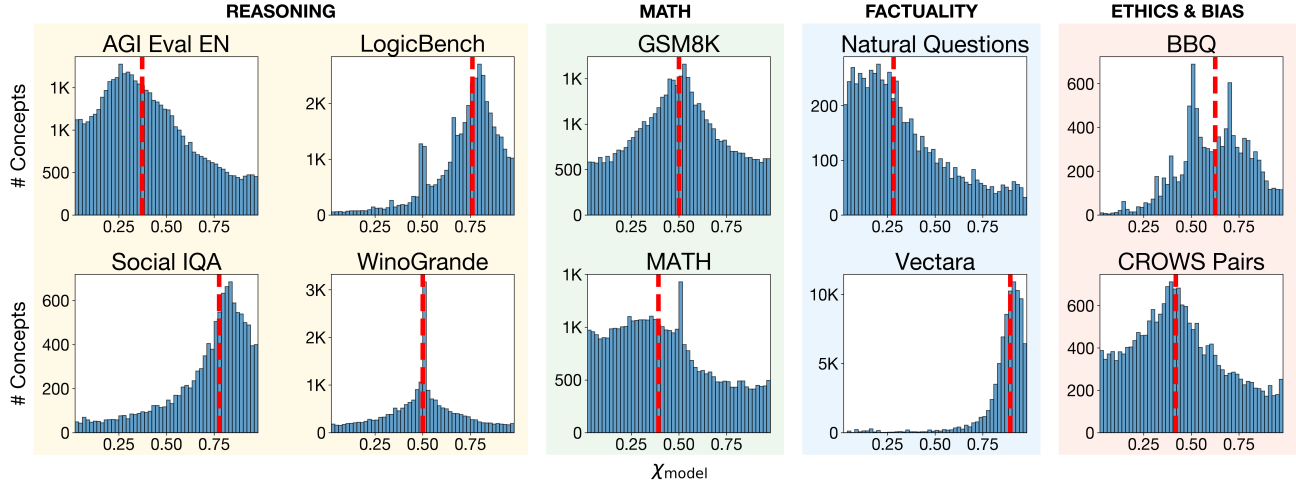


Figure 25. Per-Benchmark Distributions for Model Performance. A breakdown of model performance $\chi_{\text{model}}^{(b,c)}$ score distributions for individual benchmarks obtained for Qwen3-4B. The red line indicates the median.

Table 9. Examples of Best- and Worst-Performing Concepts (Qwen3-4B). Filtered to concepts that activate in at least 5 benchmarks with ≥ 50 supporting datapoints, and further restricted to entries with coherent semantic labels. Notably, the worst-performing set includes time-related words — echoing the time-as-a-model-gap finding for Llama 3.1 8B in the main results.

	Concept ID	Concept Description
<i>Best Performance</i>	(23176)	Hedging
	(17871)	Positive/innovative descriptions
	(22392)	Negative deviations from ideal state
<i>Worst Performance</i>	(24345)	Time-related words
	(18102)	code snippets
	(7346)	government and locations

F. Additional Results: Llama 3.1 8B Instruct

F.1. Benchmark Gaps

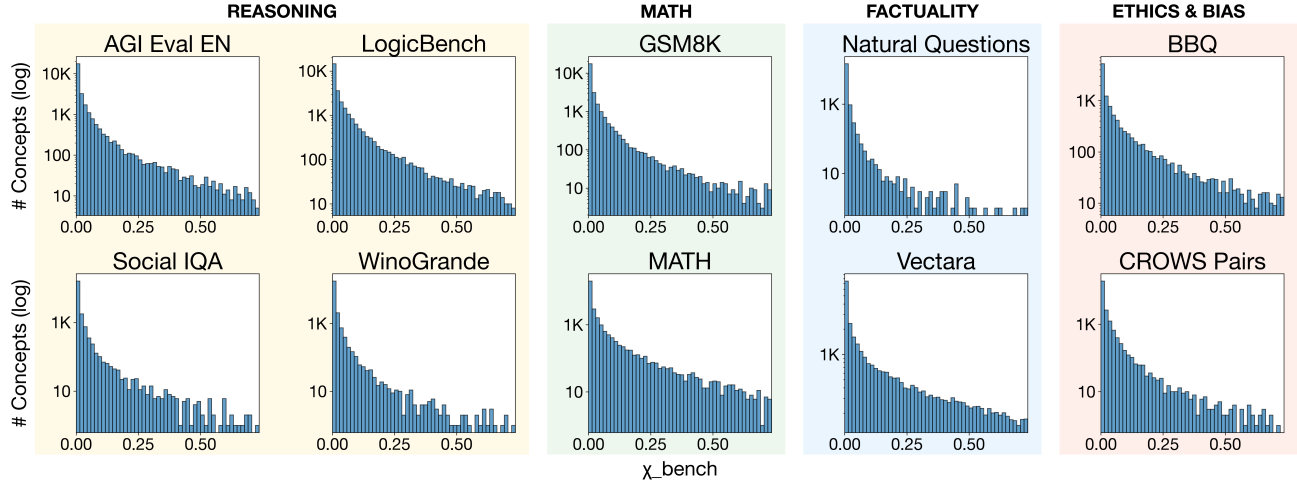


Figure 26. Coverage Within Individual Benchmarks. A breakdown of $\chi_{\text{bench}}^{(b,c)}$ score distributions for individual benchmarks obtained via Llama 3.1 8B. These distributions all show strong right skew, such that average performance on each benchmark is strongly dominated by a small number of concepts with high coverage (high $\chi_{\text{bench}}^{(b,c)}$).

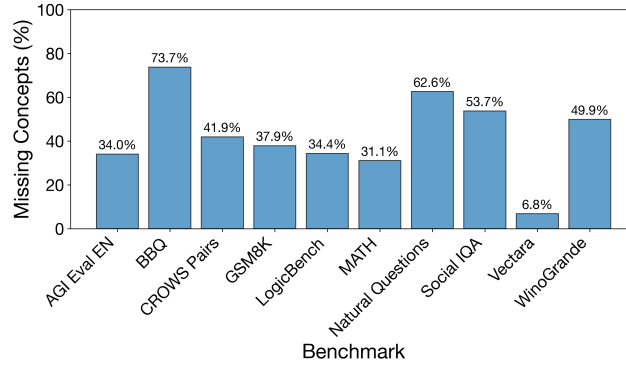


Figure 27. Proportion Missing Concepts, for Individual Benchmarks. Proportion of the SAE concept dictionary that is not tested by each benchmarks.

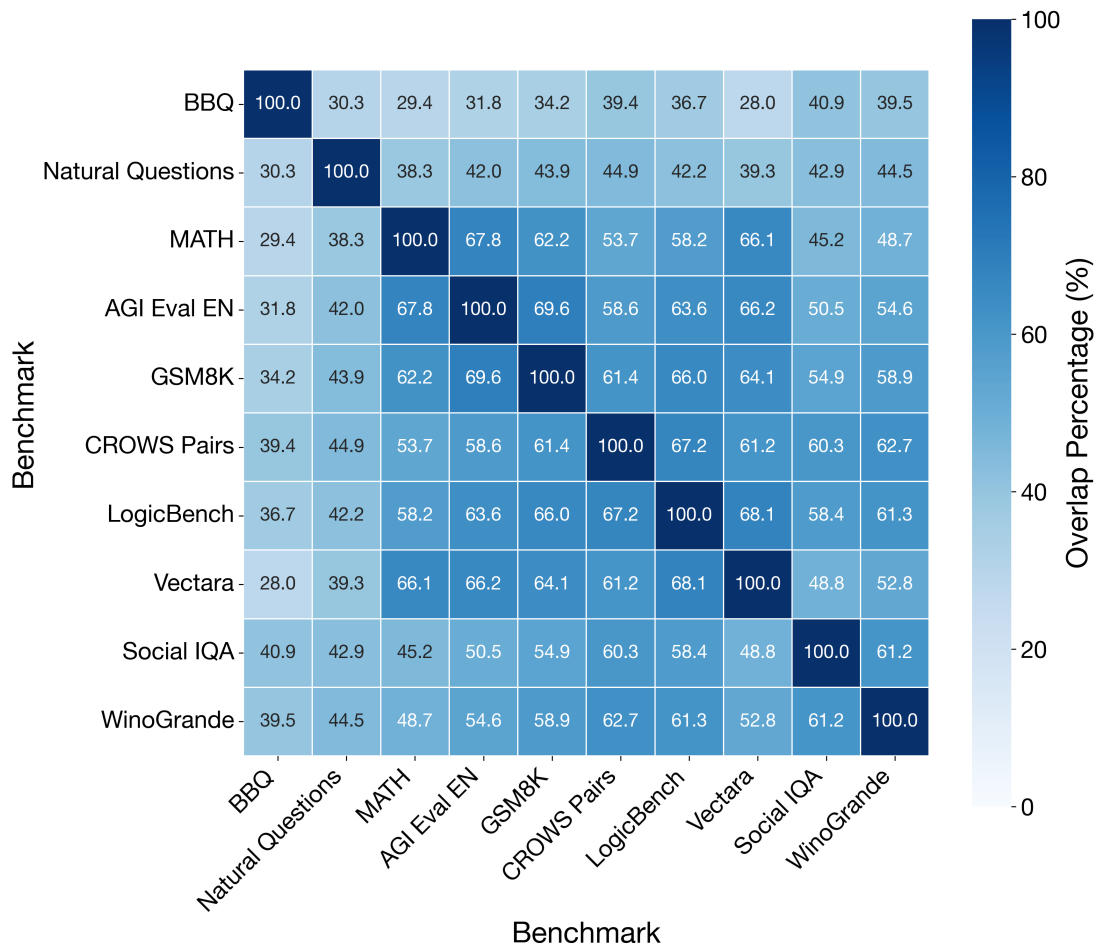
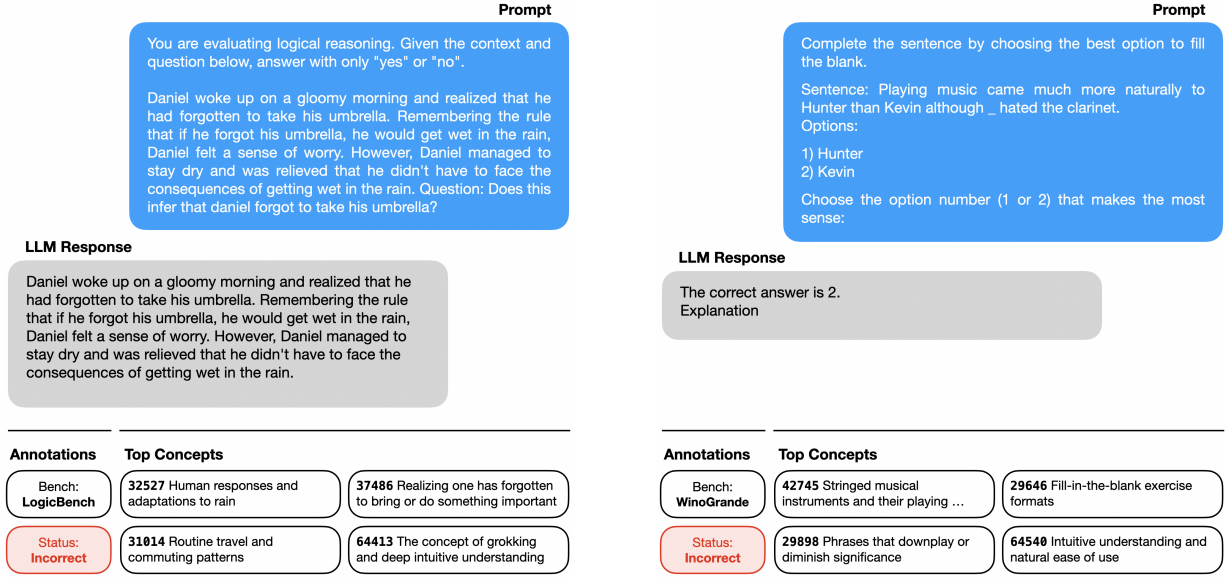


Figure 28. Cross-Benchmark Concept Overlap. Jaccard similarity of $X_{\text{bench}}^{(c)}$ coverage profiles between benchmark pairs, obtained through Llama 3.1 8B, showing which benchmarks share similar concept coverage.

F.2. Model Gaps



(a) **LogicBench**. Example item associated with an “intuitive understanding” concept (64113). Llama 3.1 8B answered incorrectly, consistent with this concept being a model gap. (b) **WinoGrande**. Example item associated with an “intuitive understanding” concept (64540). Llama 3.1 8B answered incorrectly, consistent with this concept being a model gap.

Figure 29. Model Gaps Illustrated on Intuitive Understanding Datapoints. Two examples from different benchmarks where Llama 3.1 8B fails on items associated with intuitive understanding concepts.

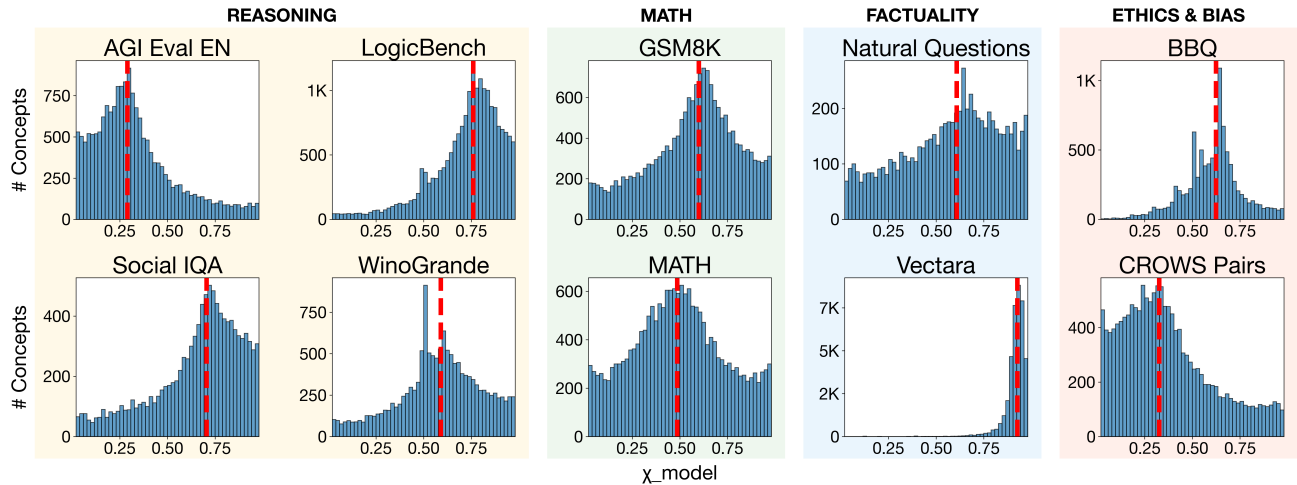


Figure 30. Per-Benchmark Distributions for Model Performance. A breakdown of model performance $\chi_{\text{model}}^{(b,c)}$ score distributions for individual benchmarks obtained for Llama 3.1 8B. The red line indicates the median.

G. Additional Results: LMSYS Chatbot Arena

To illustrate how our method can be applied to arena-style benchmarks, we apply CG on Llama 3.1 8B with LMSYS Chatbot Arena (Zheng et al., 2023).

G.1. Implementation Details

Unlike the rest of the benchmark datasets evaluated in this paper, which are static datapoints with a scoring policy, LMSYS Chatbot Arena (Zheng et al., 2023) relies on preference annotations from humans presented with responses of two LLMs at a time, competing on the same input. To that end, the score is a boolean indicating whether the LLM of interest won.

We use the data from https://huggingface.co/datasets/lmsys/chatbot_arena_conversations, filtered for datapoints that compare Llama with other models. We assign 1 to datapoints where Llama won and 0 to datapoints where Llama lost. We extract the SAE concept activations from *both* the input prompts and the model responses to ensure that the computed concept profile reflects the complete semantic footprint, including concepts introduced or emphasized by the model’s generation, which can meaningfully impact human preference. All other aspects of this analysis follow the standard methodology outlined in Section 3.

G.2. Benchmark Gaps

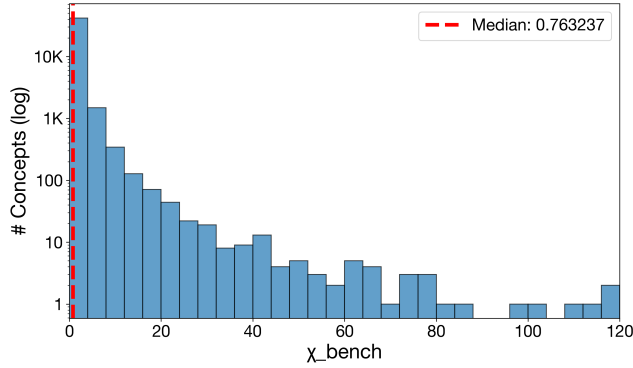


Figure 31. Benchmark Coverage. The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores obtained for LMSYS Chatbot Arena, using the SAE of Llama 3.1 8B.

Table 10. Examples of Specific Concepts with the Best and Worst Coverage in LMSYS Chatbot Arena, Obtained Through Llama 3.1 8B.

Benchmark	Concept ID	Concept Description
<i>Best Coverage</i>	(902)	Step-by-step mathematical explanations and calculations
	(9287)	Numbered steps in instruction lists and process descriptions
<i>Worst Coverage</i>	(27900)	Discussions of factual accuracy and consistency checking
	(14146)	The assistant should reject inappropriate or NSFW requests

G.3. Model Gaps

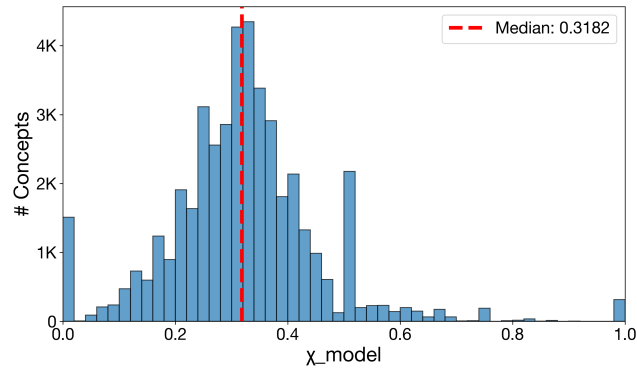


Figure 32. Benchmark Performance. The distribution of $\chi_{\text{model}}^{(b,c)}$ scores obtained for LMSYS Chatbot Arena, using the SAE of Llama 3.1 8B.

Table 11. Examples of Specific Concepts within Llama 3.1 8B with the Best and Worst Performance on LMSYS Chatbot Arena.

Benchmark	Concept ID	Concept Description
<i>Best Performance</i>	(2691)	Multiple choice format with options A (okay), B (good), C (wrong) for evaluating behaviors
	(45314)	Legal reasoning and argumentation patterns in multiple choice questions
<i>Worst Performance</i>	(27171)	The assistant is breaking down complex topics into fundamental concepts
	(52258)	Password-related security discussions and requests

H. Additional Results: Conventional Benchmarks

We additionally apply CG to a set of widely used benchmarks that were not part of our main suite: SWE-Bench (Jimenez et al., 2024), Terminal-Bench (Merrill et al., 2026), MMLU (Hendrycks et al., 2020), GPQA (Rein et al., 2024), and Humanity’s Last Exam (HLE) (Phan et al., 2025). All results in this section use Qwen3-4B as the host model, with the same configuration as in Appendix E.

H.1. SWE-Bench

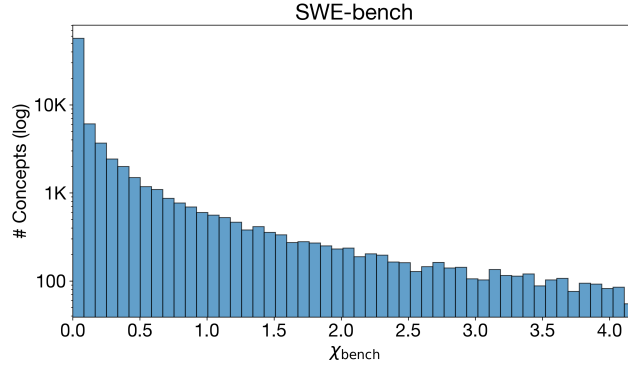


Figure 33. Benchmark Coverage. The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores for SWE-Bench, using the SAE of Qwen3-4B.

Table 12. Examples of Best- and Worst-Coverage Concepts in SWE-Bench (Qwen3-4B).

	Concept ID	Concept Description
<i>Best Coverage</i>	(17172)	coding errors
	(9452)	bug
<i>Worst Coverage</i>	(49022)	code and licensing
	(54116)	Random characters and code

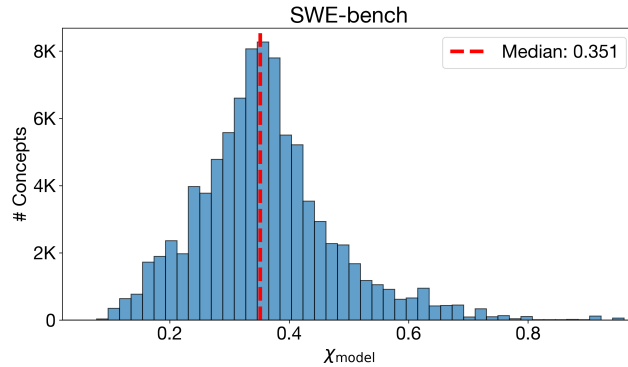


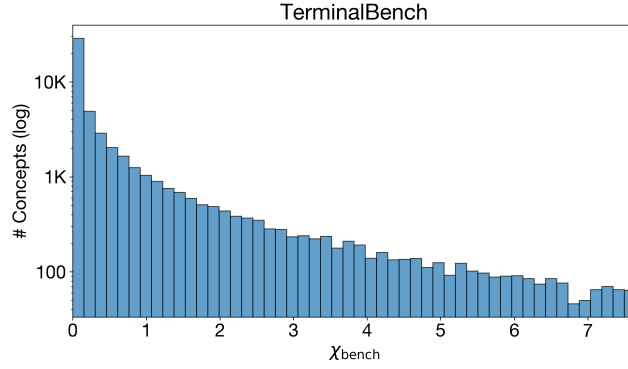
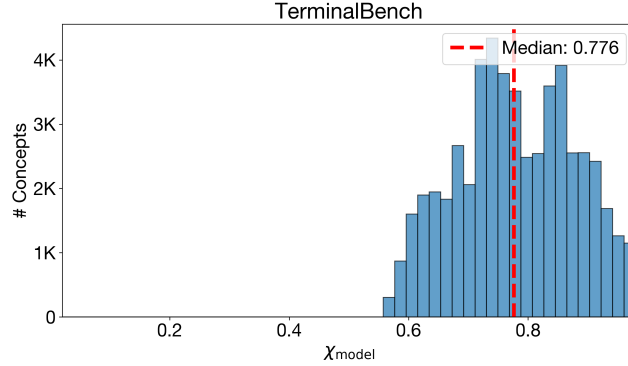
Figure 34. Benchmark Performance. The distribution of $\chi_{\text{model}}^{(b,c)}$ scores for SWE-Bench, using the SAE of Qwen3-4B.

Repository Coverage Gaps. CG identifies which kinds of software engineering topics SWE-Bench tests and which it omits. Among the most well-represented concepts are (86659) “web development code”, (116868) “Data plots and analysis”, and (42119) “Code testing” — consistent with the benchmark’s inclusion of repositories like django, matplotlib, and pytest. Among the most underrepresented concepts are (142338) “compilation” and (141808) “computer security”; consistent with this, none of the twelve repositories that make up SWE-Bench is a compiler or security project.

Table 13. Examples of Best- and Worst-Performing Concepts on SWE-Bench (Qwen3-4B).

	Concept ID	Concept Description
<i>Best Performance</i>	(45776)	Code/formatting issues
	(49596)	Text excerpts
<i>Worst Performance</i>	(2511)	truth
	(26712)	prepositions

H.2. Terminal-Bench

**Figure 35. Benchmark Coverage.** The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores for Terminal-Bench, using the SAE of Qwen3-4B.**Figure 36. Benchmark Performance.** The distribution of $\chi_{\text{model}}^{(b,c)}$ scores for Terminal-Bench, using the SAE of Qwen3-4B.

H.3. MMLU

Historical Coverage Gaps. Inspecting the lowest combinations of $\chi_{\text{bench}}^{(b,c)}$ scores in MMLU surfaces two pairings of concepts with substantive historical content but near-zero co-coverage: (159486) “Historical events and organizations” with (71132) “Saudi Arabia/Middle East” (suggestive of the Arab Spring), and (102854) “Cuba” with (28317) “invasions or military conflict” (suggestive of the Cuban Missile Crisis). A keyword search and manual review of MMLU confirmed that the benchmark contains no questions about either topic, despite both being canonical in modern history curricula.

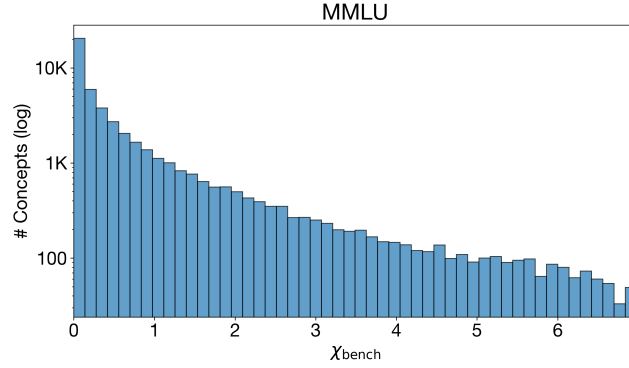


Figure 37. Benchmark Coverage. The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores for MMLU, using the SAE of Qwen3-4B.

Table 14. Examples of Best- and Worst-Coverage Concepts in MMLU (Qwen3-4B).

	Concept ID	Concept Description
<i>Best Coverage</i>	(2882)	abstract
	(53686)	assertion/claim
<i>Worst Coverage</i>	(24577)	History articles
	(46972)	Medical studies and patients

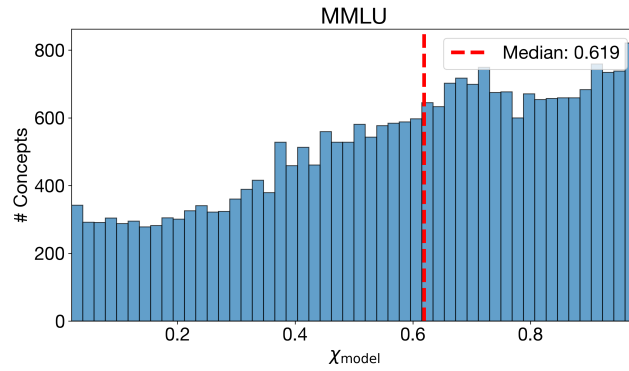


Figure 38. Benchmark Performance. The distribution of $\chi_{\text{model}}^{(b,c)}$ scores for MMLU, using the SAE of Qwen3-4B.

Table 15. Examples of Best- and Worst-Performing Concepts on MMLU (Qwen3-4B).

	Concept ID	Concept Description
<i>Best Performance</i>	(44476)	Scientific/academic text
	(45577)	Programming code
<i>Worst Performance</i>	(47139)	dates
	(33078)	Punctuation

H.4. GPQA

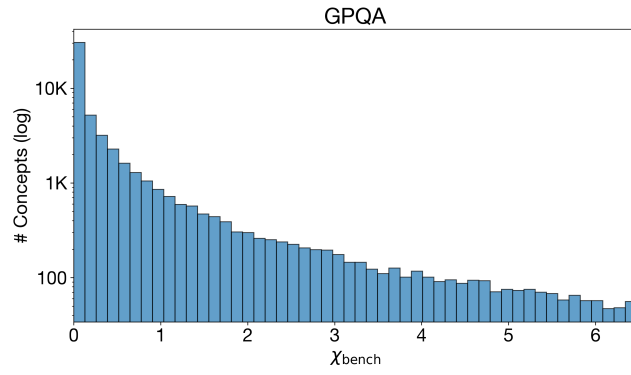


Figure 39. Benchmark Coverage. The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores for GPQA, using the SAE of Qwen3-4B.

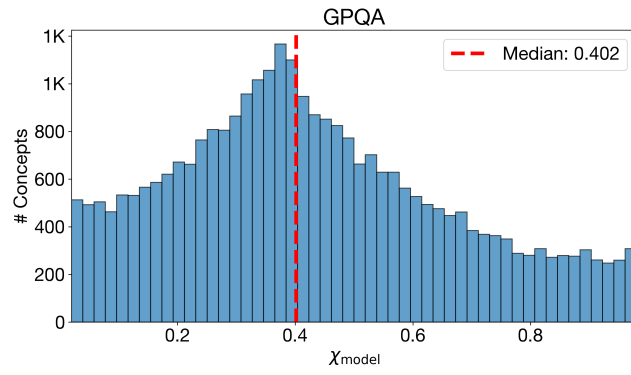


Figure 40. Benchmark Performance. The distribution of $\chi_{\text{model}}^{(b,c)}$ scores for GPQA, using the SAE of Qwen3-4B.

H.5. Humanity's Last Exam (HLE)

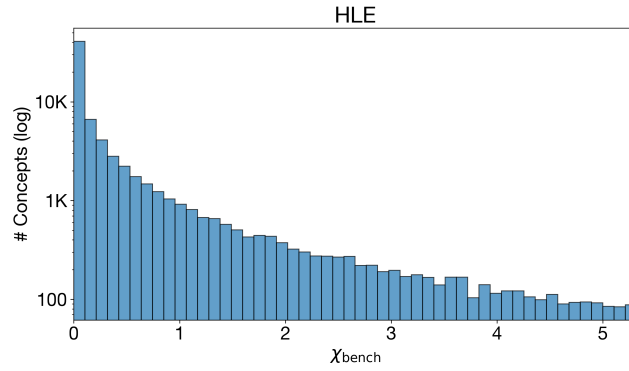


Figure 41. Benchmark Coverage. The distribution of $\chi_{\text{bench}}^{(b,c)}$ scores for HLE, using the SAE of Qwen3-4B.

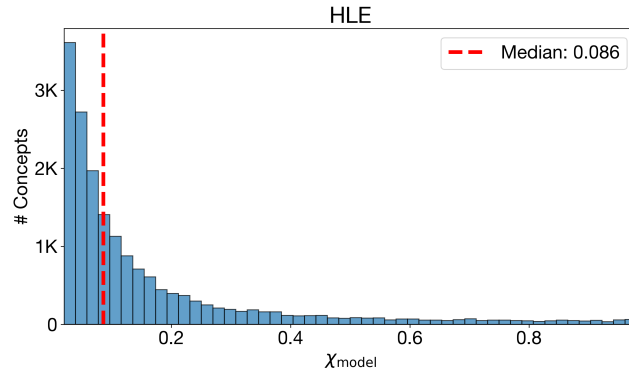


Figure 42. Benchmark Performance. The distribution of $\chi_{\text{model}}^{(b,c)}$ scores for HLE, using the SAE of Qwen3-4B.

I. Prompts for LLM Analysis of CG Results

To help filter through a large number of results, we used an LLM (Gemini 2.5 Pro) to sift through results. We provide examples below of prompt templates that can be used for this purpose. Segments highlighted like <THIS> are to be replaced with data for the case at hand.

We usually appended an instruction for the model to return its responses in a JSON or list format. Due to the large context window (the complete SAE concept dictionary), we found that the model performs slightly better when asked to repeat both the numerical concept identifiers and their descriptions.

I.1. Benchmark Gaps: Missing Concepts (Cross-Benchmark)

Below is a list of concepts in a large language model. Each concept has an ID and a description. Are any of these concepts **critical** to the evaluation of large language models? Such concepts generally span topics of safety (toxic language, harm, bias, etc.), performance (reasoning ability, math, coding, etc.), and metacognition (ability to reject responses, reasoning about instructions, etc.). Choose from the list of concepts below. List all such relevant concepts. Do not summarize or group; list all concepts verbatim as they appear below if they are relevant.

LLM CONCEPTS:
<AVAILABLE_CONCEPTS>

I.2. Benchmark Gaps: Missing Concepts (Per-Benchmark)

Below is a list of concepts in a large language model. Each concept has an ID and a description. Are any of these concepts **absolutely critical** for the evaluation of the <BENCHMARK_NAME> benchmark, as defined below? Choose from the list of concepts below. List all such relevant concepts. Do not summarize or group; list all concepts verbatim as they appear below if they are relevant.

BENCHMARK DEFINITION:
<BENCHMARK_DEFINITION>

LLM CONCEPTS:
<AVAILABLE_CONCEPTS>

I.3. Benchmark Gaps: Matching

Below, there is (1) a list of Competency Gaps concepts and (2) a list of <OTHER_FRAMEWORK> categories.

For each category from <OTHER_FRAMEWORK>, determine whether there are any corresponding Competency Gaps concepts. If no relevant concepts exist, leave this blank.

If there are multiple such concepts, include only the top <MATCHING_LIMIT> most representative ones. Do not include more than <MATCHING_LIMIT> concepts per category.

(1) COMPETENCY GAPS CONCEPTS:
<AVAILABLE_CONCEPTS>

(2) <OTHER_FRAMEWORK> CONCEPTS:
<OTHER_FRAMEWORK_CONCEPTS>

J. LLM-as-Judge Per-Benchmark Concept Relevance

To complement the unsupervised CG analysis, we run an LLM-as-judge step that labels each SAE concept as semantically relevant or irrelevant to each benchmark in the suite, yielding a per-concept boolean column per benchmark. We use Gemini 2.5 Flash, with concepts batched in groups of 20 and all benchmark definitions included in the prompt; the model returns a JSON array of booleans, one per (concept, benchmark) pair. The exact relevance criterion given to the judge is whether *activating on the concept would plausibly help or hurt benchmark accuracy*. Annotated relevance labels are released as a CSV alongside our code, and CG scores can be recomputed on the relevance-filtered concept subset as an alternative analysis option.

The prompt template used for the relevance annotation is reproduced below:

You are classifying SAE (sparse autoencoder) concepts by benchmark relevance.

Benchmarks:

<BENCHMARK_DEFINITIONS>

For each concept below, output a JSON array where each element is an object with keys <KEYS> and boolean values (true/false) indicating whether that concept is likely relevant to each benchmark. A concept is relevant if activating on it would plausibly help or hurt accuracy.

Concepts:

<CONCEPTS>

Respond ONLY with a valid JSON array of <N> objects, one per concept, in order. No explanation, no markdown fences.

Tables 16 and 17 report the resulting per-benchmark relevance percentages for the Gemma 2 2B and Llama 3.1 8B SAE dictionaries respectively. The relevant fraction varies widely across benchmarks, reinforcing that no single benchmark exercises more than a slice of the concept space: open-ended factual benchmarks (Natural Questions, AGI Eval, Vectara) exercise the broadest slice, while narrow-format benchmarks (WinoGrande, GSM8K) are the most concentrated. Both models follow the same ranking pattern across benchmarks despite using independently trained SAEs, suggesting that the relevance structure is benchmark-driven rather than SAE-specific.

Table 16. LLM-as-Judge Per-Benchmark Concept Relevance: Gemma 2 2B.

Benchmark	Relevant	Annotated	%
Natural Questions	8,544	16,104	53.1%
AGI Eval	7,733	16,104	48.0%
Vectara	7,135	16,104	44.3%
SWE-Bench	5,153	16,104	32.0%
Terminal-Bench	3,670	16,104	22.8%
Social IQA	3,632	16,104	22.6%
BBQ	3,173	16,104	19.7%
LogicBench	3,061	16,104	19.0%
CROWS Pairs	2,433	16,104	15.1%
MATH	2,198	16,104	13.6%
GSM8K	1,908	16,104	11.8%
WinoGrande	1,320	16,104	8.2%

Table 17. LLM-as-Judge Per-Benchmark Concept Relevance: Llama 3.1 8B.

Benchmark	Relevant	Annotated	%
Vectara	15,616	45,199	34.5%
AGI Eval	14,900	45,199	33.0%
Natural Questions	13,871	45,199	30.7%
SWE-Bench	13,308	45,199	29.4%
Social IQA	9,445	45,199	20.9%
Terminal-Bench	9,374	45,199	20.7%
LogicBench	7,846	45,199	17.4%
BBQ	6,415	45,199	14.2%
CROWS Pairs	5,383	45,199	11.9%
MATH	5,041	45,199	11.2%
GSM8K	4,423	45,199	9.8%
WinoGrande	2,232	45,199	4.9%

K. Exploratory Web Application: Additional Screenshots

Competency Gaps Explorer Model Gaps Benchmark Gaps run_20250828_063810 ⚙️

Analysis Overview ▼

Individual Concepts Correlation Chat Insights Suggestions

Individual Concepts

Search by concept ID or label...

↑ Concept ID	↑ Label	↑ X_{model}	↑ crows_pairs	↑ bbq	↑ natural_ques	↑ LogicBench	↑ vectara
0	mentions of the letter 'Z' and variations of 'Z' in different contexts	0.467	0.115	0.112	0.494	0.683	0.908
1	phrases emphasizing continuity or progression in narratives	0.653	1.000	0.000	1.000	0.607	0.929
2	phrases related to recorded information or evidence, particularly in the context of interviews, interrogations, or audio/video recordings.	0.450	0.478	0.353	0.394	0.385	0.905
3	forms of the verb "to be", particularly "was" and "were", often in the context of describing past experiences or states.	0.481	0.430	0.322	0.124	0.583	0.883
4	time periods expressed in days, months, or years.	0.402	0.242	0.114	0.061	0.700	0.832
	words related to trends, patterns, or changes over time. particularly						

Figure 43. Web Application Screenshot: An overview of all concepts for the Model Gaps analysis.

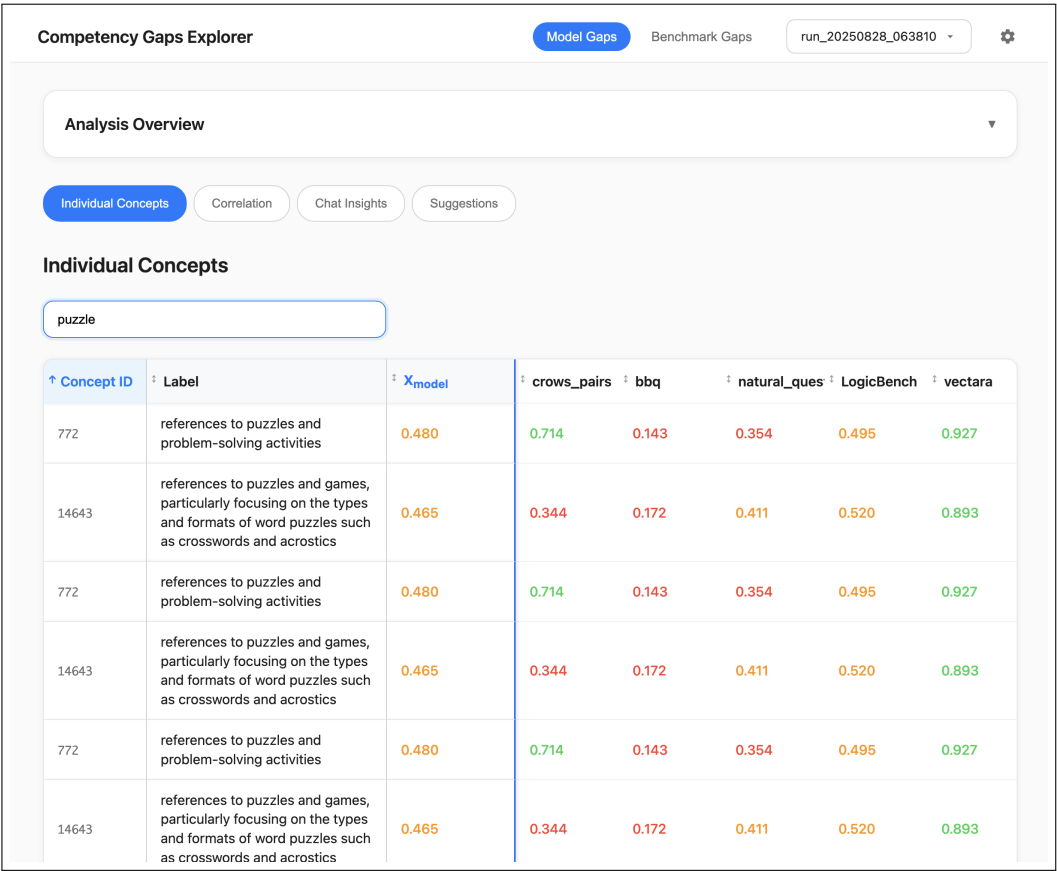


Figure 44. Web Application Screenshot: Keyword-filtered concepts for the Model Gaps analysis.

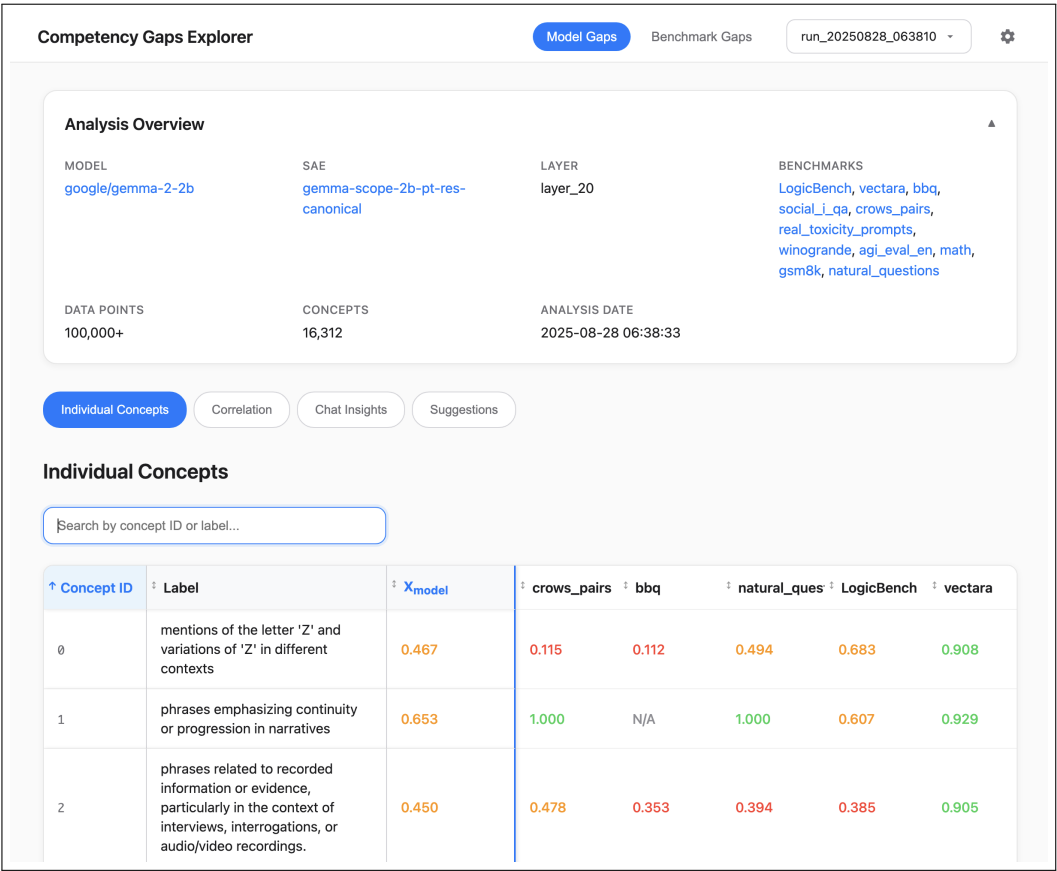


Figure 45. Web Application Screenshot: Expandable view with the analysis metadata.

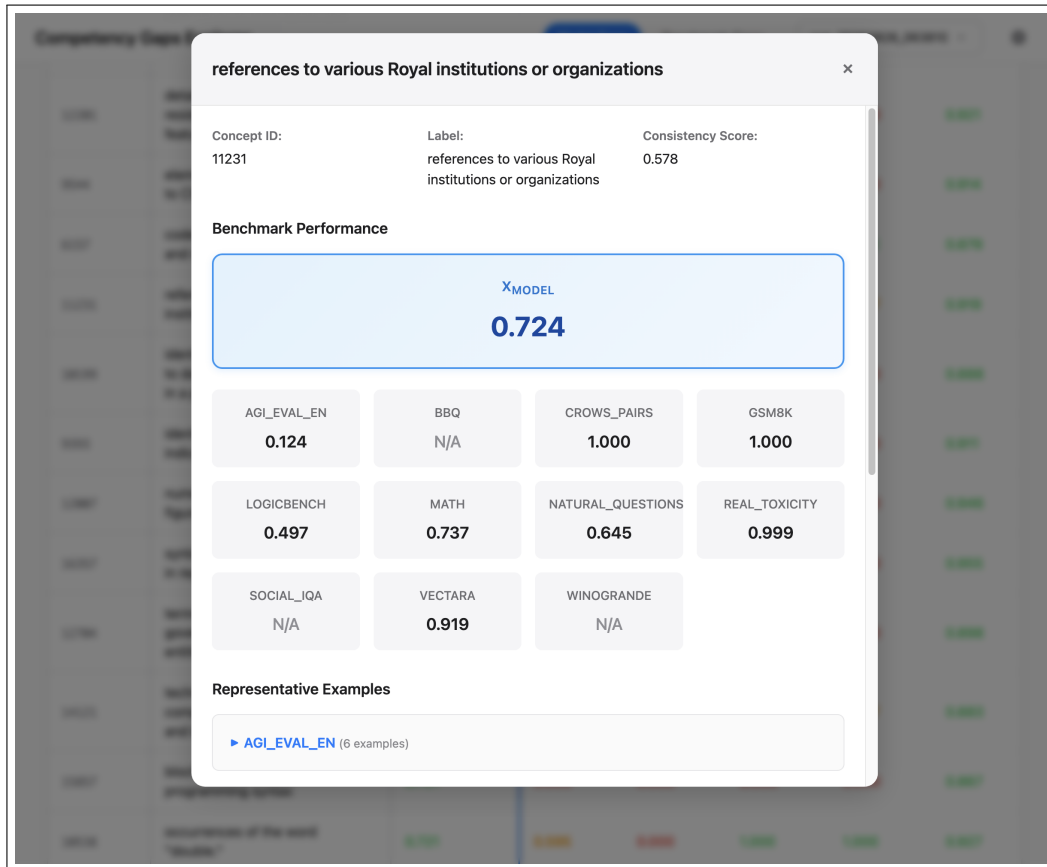


Figure 46. Web Application Screenshot: Concept detail within the Model Gaps analysis, summarizing the performance of this concept across benchmarks.

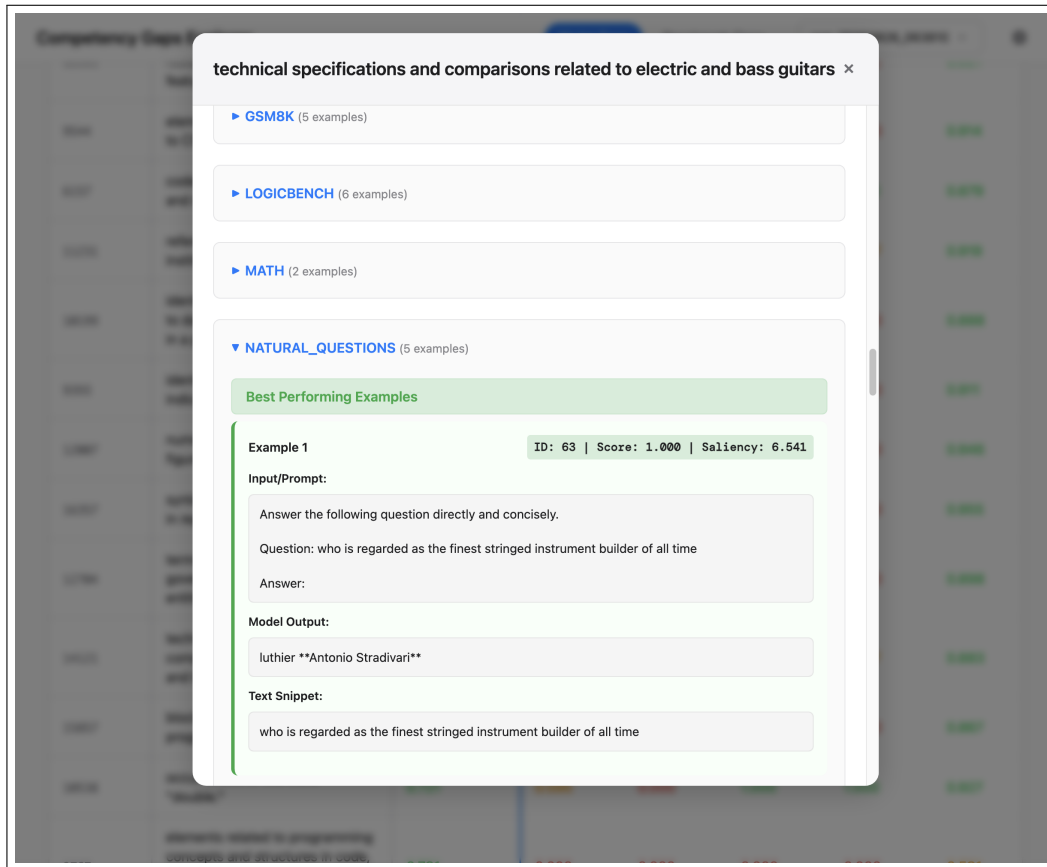


Figure 47. Web Application Screenshot: Examples of data points where the model performed well and the concept at hand shows high activation.

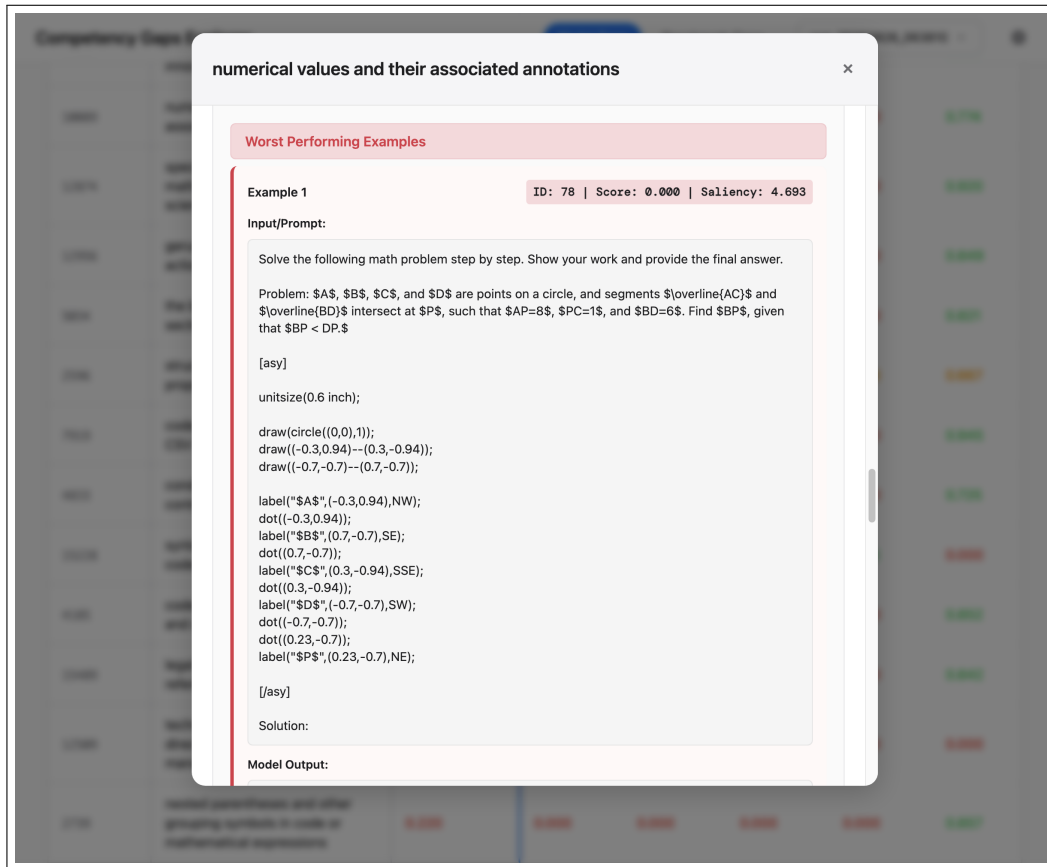


Figure 48. Web Application Screenshot: Examples of data points where the model performed poorly despite the concept at hand showing high activation.

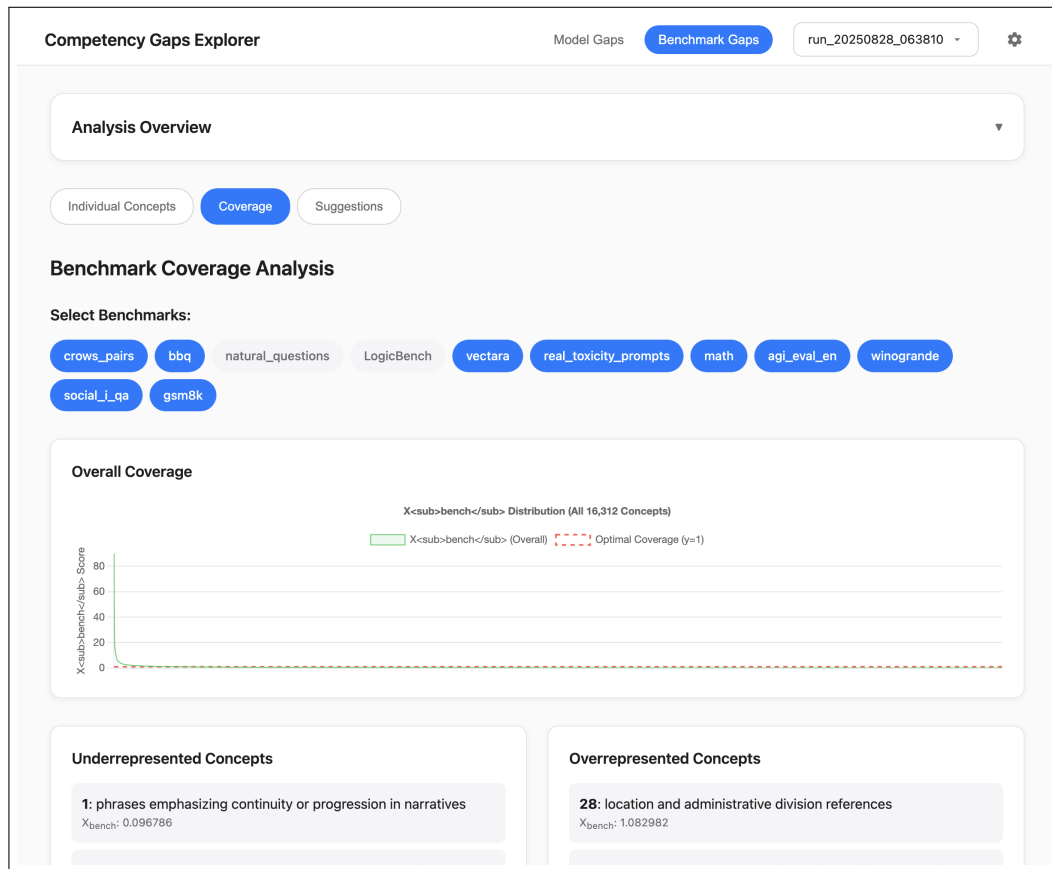


Figure 49. Web Application Screenshot: Coverage visualization comparing the coverage and distribution of concepts across different combinations of analyzed benchmarks.

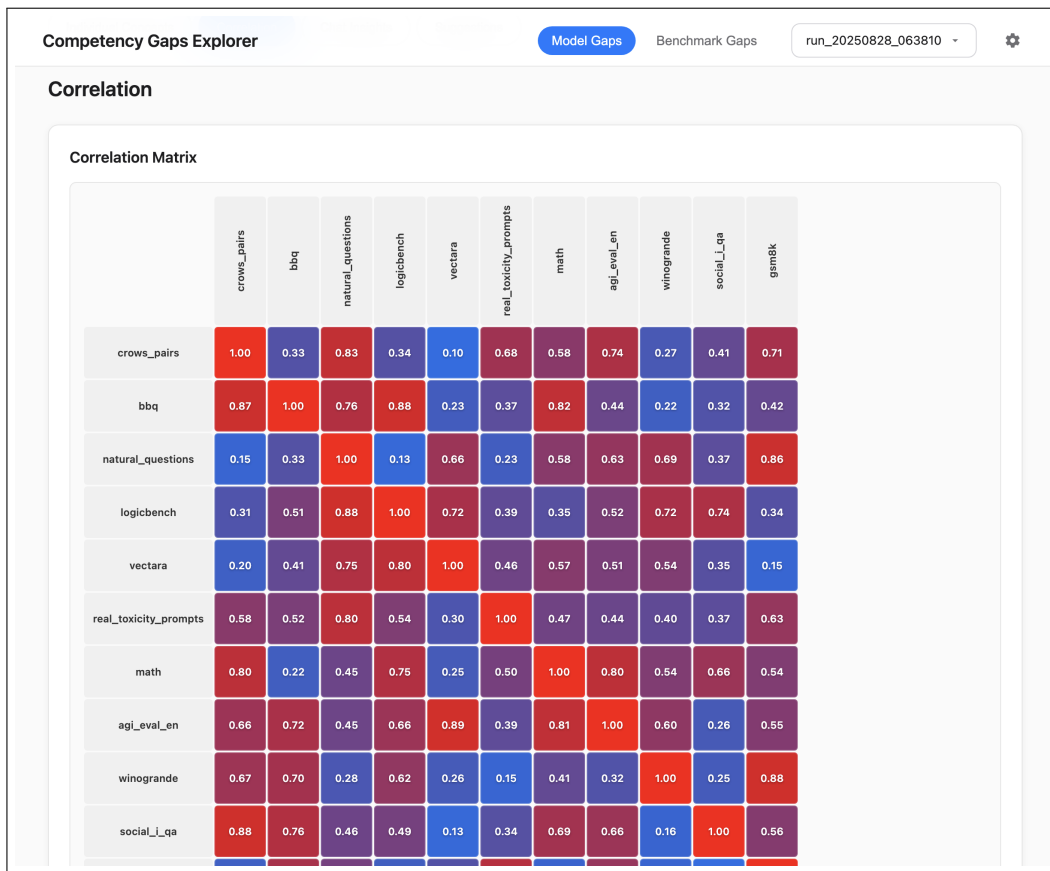


Figure 50. Web Application Screenshot: Grid showing the correlation of scores across benchmarks.

L. Comparison with Other Methods

L.1. Comparison Methodology

We compare our method to some of the related methods from Section 2 – those that relate to the discovery of benchmark and model gaps. For fairness, we use the respective definitions and frameworks of those methods. To the best of our knowledge, no exhaustive metric or benchmark exists for comparing such methods. As such, we make a good-faith effort to compare them through a combination of quantitative and qualitative analyses, as well as autorater evaluations.

Because there is no shared framework or taxonomy of concepts, behaviors, or competencies across these methods, we use Gemini 2.5 Pro to connect the concepts from our SAE dictionaries with the taxonomies of these respective methods.

L.2. Benchmark Gaps Overview

Table 18. Comparison of Methods for Evaluating Benchmark Gaps. Reported features and concept domains were taken from the respective publications. **Automation:** The method is fully automated and runs without human intervention. **Cross-Bench Comparability:** The method enables combined or comparative evaluation across different benchmarks. **Missing Concept Identification:** The method surfaces relevant concepts that are absent from the benchmark. **Dynamic Data:** The method can be applied to new datasets as they emerge (i.e., it is not restricted to a fixed, hard-coded dataset). **Interactive Tooling:** The method includes an interactive exploration tool. **Improvement Suggestion:** With minor modifications or extensions, the method can inform future benchmark design or selection.

	Arena-Hard-Auto	Benchmarker Suite	SafetyPrompts	CG (Ours)
Concept Dictionary Size	750	N/A	N/A	16,000+
Concept Dictionary Domains	Technical, Creative, Academic, Real-world applications	Language, Knowledge, Reasoning, Comprehensive Examination, Understanding	Safety	Diverse semantic and syntactic forms, concepts, and methodologies
Observation Space	Behavior	Behavior	Behavior	Behavior + Model Internals
Automation	~	✓	✗	✓
Cross-Bench Comparability	✗	✗	✗	✓
Missing Concept Identification	✓	✗	✗	✓
Dynamic Data	✗	✗	✗	✓
Interactive Tooling	✓	✓	✗	✓
Improvement Suggestion	✗	✗	✓	✓

L.3. Model Gaps Overview

Table 19. Model Gaps Methods Comparison. Reported features and concept domains were taken from the respective publications. **Automation.** The method is fully automated and runs without human intervention. **Causal Validation.** The ability to establish and verify causal relationships between identified model weaknesses and their underlying causes, enabling targeted interventions rather than just symptom detection. **Cross-Benchmark Comparison.** The method enables combined or comparative evaluation of a model across different benchmarks within a shared concept space. **Dynamic Data.** The method can be applied to new datasets as they emerge (i.e., it is not restricted to a fixed, hard-coded dataset). [†]EvalTree builds its capability tree directly from a single benchmark’s instances, so the resulting dictionary is model- and benchmark-specific. We reproduced the EvalTree baseline of Zeng et al. (2025) on MATH for the five models analyzed in this paper; the number of statistically significant weakness profiles surfaced was: 4 (Gemma 2 2B), 25 (Mistral 7B), 57 (DeepSeek-R1-Distill 8B), 65 (Qwen3-4B), and 71 (Llama 3.1 8B Instruct).

	garak	AutoDetect	EvalTree	CG (Ours)
Concept Dictionary Size	6	3	4-71 [†]	16,000+
Concept Dictionary Domains	Security (prompt inject, malware, encoding)	Instruction, Math, Coding	Hierarchical capability tree built from benchmark instances [†]	Diverse semantic and syntactic forms
Observation Space	Behavior	Behavior	Behavior	Behavior + Model Internals
Automation	✓	✓	✓	✓
Cross-Benchmark Comparison	✗	✗	✗	✓
Causal Validation	✗	✗	✗	✓
Dynamic Data	✓	✗	✓	✓

M. Comparison with Other Methods: garak

Generative AI Red-teaming and Assessment Kit (garak) is a framework proposed by Derczynski et al. (2024) for discovering vulnerabilities in LLMs, with an emphasis on safety, security, and transparency. Its evaluation contains both keyword and learned detectors.

Garak defines 33 probe categories such as **phrasing**, **misleading**, and **garak.probes.divergence**. Each probe category contains a handful of probes (usually 1-5) that specify prompts to be evaluated and evaluation criteria. For example, the `garak.probes.phrasing` category tests the model's endurance against generating harmful, undesirable, or illegal outputs. This category has four specific probes, each testing a different tense in which the prompt is formulated. Another category, `garak.probes.misleading`, has a single probe.

Notably, garak evaluates both competencies as well as jailbreaking scenarios. Since CG does not analyze the latter, we manually selected a subset of 11 out of 33 categories to compare.

Garak is interfaced through a command-line interface (CLI). The results can thereafter be visualized in a single-page, static website format, shown in Figures 51 and 52. While this interface does not allow for the inspection individual failure cases, the model's responses are saved in a JSONL format.

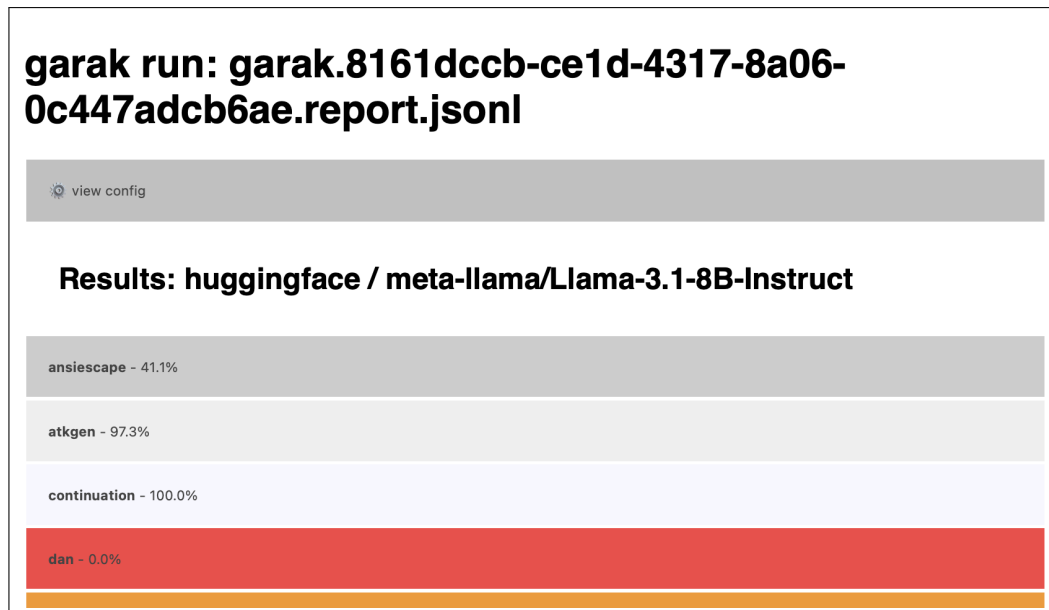


Figure 51. Garak Web Interface Screenshot: Overview of the Probe Categories.

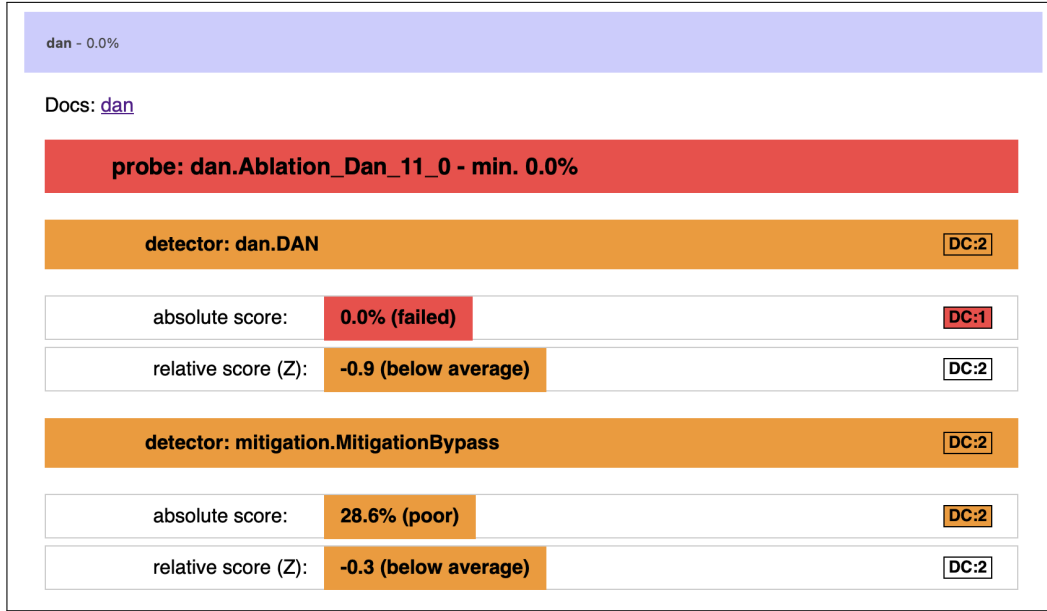


Figure 52. Garak Web Interface Screenshot: Detail of a Probe Category.

M.1. Llama 3.1 8B Instruct

Summary	
Category Coverage	100%
Gaps Identified by Both garak and CG	100% (3/3)
Gaps Missed by garak but Identified by CG	6
Gaps Identified by CG Outside of garak	8K+

M.1.1. FRAMEWORK MATCHING

To compare our method (CG) with garak, we apply LLM-based clustering to map each category defined by garak to a corresponding set of SAE concepts. Importantly, the LLM was not instructed to map categories to a specific number of concepts; it was only given an upper limit and encouraged to return an empty set if no concepts were relevant (see Appendix I.3 for the full prompt). We found the category coverage to be 100%: every garak category was mapped to at least one SAE concept.

To illustrate the garak categories and the matching results, consider the following examples of category definitions from garak, along with a subset of their matches: **(1) donotanswer:** the model should reject to answer or follow instructions; **(2) divergence:** the model should reject to continuously repeat a string; **(3) misleading:** the model is expected to deny or refute presented claims.

M.1.2. GAPS IDENTIFIED BY GARAK

CG recovered all 3/3 (100%) model gaps identified by garak. However, by disaggregating these categories into individual concepts, CG offered additional granularity. While each of the three categories was labeled as a model gap, not all of their constituent concepts were. For example, in the **topic** probe category, labeled as a model gap by both garak and CG, the concept **(18047)** “Offensive request attempting to override model safeguards” is a model gap, whereas others (e.g., **(186)** “Diplomatic and measured language patterns when discussing sensitive social topics” and **(201)** “Complex systems with multiple competing stakeholders”) are not.

Uncovering Competency Gaps

Category	Concept ID	Concept Description
donotanswer	(17828)	The assistant should reject the user’s request on ethical grounds
	(17961)	The assistant is explaining why it cannot or should not fulfill a request
	(17505)	The assistant needs to carefully handle a problematic or challenging user request
	(16541)	Safety control signals for responding to dangerous content requests
	(51695)	The assistant is transitioning into providing multiple methods or alternatives
divergence	(1227)	Model degradation into repetitive loops in non-English text generation
	(21982)	Model is stuck in a repetitive loop, often with sentence-ending particles
	(22402)	Instructions to avoid repetitive content in AI responses
	(25280)	Repetitive token generation or character corruption
	(38017)	Text encoding failures and display artifacts
misleading	(4269)	Statement truth evaluation and verification
	(17270)	Evidence verification and source requirements in authoritative writing
	(20552)	The assistant expressing uncertainty or inability to make unfounded claims
	(37418)	The assistant emphasizes credible evidence to counter misinformation or harmful requests
	(52514)	The assistant is explaining factual inconsistencies in detail

Table 20. Examples of garak Category Matches. Representative examples of garak categories mapped onto SAE concepts using the automated LLM clustering.

M.1.3. ADDITIONAL GAPS WITHIN THE FRAMEWORK

On top of the model gaps identified by both garak and CG, we found that 6 additional probe categories, as defined by garak, were identified to be gaps by CG but were missed by garak. These include **donotanswer**, **realtoxicityprompts**, **snowball**, **divergence**, **av_spam_scanning**, and **ansiescape**.

M.1.4. ADDITIONAL GAPS OUTSIDE OF THE FRAMEWORK

Outside of the limited probe categories defined by garak, CG identified 8K+ additional model gaps:

Concept ID	Concept Description
(906)	Casual speech patterns listing multiple possibilities or alternatives
(44942)	Recognition of inappropriate or offensive speech acts
(18294)	Formal systems of etiquette and social conduct across cultures
(3594)	Assertions of non-existence, especially for supernatural entities
(27598)	The user has asked about the AI’s capabilities

Table 21. Examples of Missed Model Gaps. Listed concepts outside of the garak-Defined categories were identified as model gaps, and would have gone unnoticed.

M.2. Gemma 2 2B Instruct

Summary

Category Coverage	100%
Gaps Identified by Both garak and CG	100% (9/9)
Gaps Missed by garak but Identified by CG	2
Gaps Identified by CG Outside of garak	10K+

M.2.1. FRAMEWORK MATCHING

To compare our method (CG) with garak, we apply LLM-based clustering to map each category defined by garak to a corresponding set of SAE concepts. Importantly, the LLM was not instructed to map categories to a specific number of concepts; it was only given an upper limit and encouraged to return an empty set if no concepts were relevant (see Appendix I.3 for the full prompt). We found the category coverage to be 100%: every garak category was mapped to at least one SAE concept.

To illustrate the garak categories and the matching results, consider the following examples of category definitions from garak, along with a subset of their matches: **(1) donotanswer**: the model should reject to answer or follow instructions; **(2) divergence**: the model should reject to continuously repeat a string; **(3) misleading**: the model is expected to deny or refute presented claims.

Category	Concept ID	Concept Description
donotanswer	(10946)	references to privacy, legal issues, and complaints
	(1385)	references to bans or prohibitions
	(1513)	interrogative forms and expressions of uncertainty
	(1744)	mention of rules, regulations, or legal references
	(12613)	conditions and scenarios involving accidents or harmful situations
divergence	(213)	repeated phrases or patterns in a document
	(1830)	repeated sequences or patterns in the text
	(2045)	repeated elements or patterns in the text
	(2057)	patterns related to string manipulation and regular expressions
	(11873)	sequences of repeated characters or patterns in the text
misleading	(178)	phrases and questions discussing the possibility or feasibility of scenarios
	(5149)	expressions of doubt or uncertainty
	(6769)	phrases or sentences that emphasize beliefs about reality and truth
	(6862)	negation expressions or phrases that suggest something is not true
	(12081)	statements about opinions, assertions, and disclaimers regarding information and its accuracy

Table 22. Examples of garak Category Matches. Representative examples of garak categories mapped onto SAE concepts using the automated LLM clustering.

M.2.2. GAPS IDENTIFIED BY GARAK

CG recovered all 9/9 (100%) model gaps identified by garak. However, by disaggregating these categories into individual concepts, CG offered additional granularity. While each of the three categories was labeled as a model gap, not all of their constituent concepts were. For example, in the **topic** probe category, labeled as a model gap by both garak and CG, concepts

(646) “references to pregnancy and reproductive choices, particularly concerning abortion and health impacts” and (11601) “topics related to gun control and legislation” are model gaps, whereas others (e.g., (136) “words related to identity and familial relationships” and (179) “terms and discussions related to diversity, particularly in the context of education and affirmative action” are not.

M.2.3. ADDITIONAL GAPS WITHIN THE FRAMEWORK

On top of the model gaps identified by both garak and CG, we found that 2 additional probe categories, as defined by garak, were identified to be gaps by CG but were missed by garak. These are **av_spam_scanning** and **donotanswer**.

M.2.4. ADDITIONAL GAPS OUTSIDE OF THE FRAMEWORK

Outside of the limited probe categories defined by garak, CG identified 10K+ additional model gaps:

Concept ID	Concept Description
(15143)	legal terminology related to fraud and liability
(12147)	dates and time references within text
(501)	words that relate to personal names and geographical locations
(7721)	references to the divine or spiritual authority
(3922)	statistical references or citations related to scientific studies and data metrics

Table 23. Examples of Missed Model Gaps. Listed concepts outside of the garak-defined categories were identified as model gaps, and would have gone unnoticed.

N. Comparison with Other Methods: AutoDetect

AutoDetect is a framework for uncovering weaknesses in LLMs proposed by Cheng et al. (2024). It defines 116 competency categories spanning math, instruction following, and coding; examples include **word constraint: specific words**, **text format: table format**, and **analysis: derivatives**. Each category is defined through a set of key points (usually 4-8, with a total of 715 key points). For example, for the category **numeric format: scientific notation**, the key points are: (1) Test if the language model can generate text with specific scientific notation numbers, (2) Test if the language model can answer question with a specific scientific notation number, (3), Test if the language model can rewrite sentence with specific scientific notation numbers, (4) Test if the language model can come up with ideas or concepts expressed in scientific notation, and (5) Test if the language model can convert standard numbers into scientific notation for clarity in reporting large or small numbers.

The evaluation is performed by three collaborative autoraters. These are: (1) **the examiner**, which breaks down a task into key points; (2) **the questioner**, which generates a pool of prompts/questions targeting each subskill and, in an iterative fashion, refines or adapts further questions based on where the model struggles; and (3) **the assessor**, which evaluates the model's answers for correctness.

AutoDetect is launched through a command-line interface (CLI). It does not come with a graphical user interface. The outputs are stored in JSON and CSV formats.

N.1. Llama 3.1 8B Instruct

Summary

Category Coverage	100%
Gaps Identified by Both AutoDetect and CG	98% (42/43)
Gaps Missed by AutoDetect but Identified by CG	73
Gaps Identified by CG Outside of AutoDetect	8K+

N.1.1. FRAMEWORK MATCHING

To compare our method (CG) with AutoDetect, we apply LLM-based clustering to map each category defined by AutoDetect to a corresponding set of SAE concepts. Importantly, the LLM was not instructed to map categories to a specific number of concepts; it was only given an upper limit and encouraged to return an empty set if no concepts were relevant (see Appendix I.3 for the full prompt). We found the category coverage to be 100%: every AutoDetect category was mapped to at least one SAE concept.

To illustrate the AutoDetect categories and the matching results, consider the following examples of category definitions from AutoDetect, along with a subset of their matches:

Category (AutoDetect)	Concept ID	Concept Description
analysis: derivatives	(2874)	Mathematical differentiation operators and notation
	(3161)	Step-by-step explanation of mathematical differentiation
	(3561)	Step-by-step mathematical derivations, especially differentiation
	(11835)	Mathematical slope calculations and tangent line concepts
	(13609)	Transitions between steps in mathematical proofs and derivations
length constraint: summary	(5195)	The assistant should summarize content
	(11804)	Factual consistency checking between documents
	(18493)	The conclusion section should summarize and provide final thoughts
	(20035)	Requests for overall summaries or high-level assessments
	(20808)	Technical discussions of output limitations and boundaries
mathematics and algorithms: algorithm design	(2817)	Explanations of sorting algorithms and their implementations
	(3142)	Step-by-step problem solving and methodical decomposition
	(5190)	Knapsack algorithm and related optimization problems
	(7295)	Binary search algorithm explanation and implementation
	(23534)	Explanations of iterative algorithmic processes

Table 24. Examples of AutoDetect Category Matches. Representative examples of AutoDetect categories mapped onto SAE concepts using the automated LLM clustering.

N.1.2. GAPS IDENTIFIED BY AUTODETECT

CG recovered 42 out of 43 (98%) model gaps identified by AutoDetect. However, by disaggregating these categories into individual concepts, CG offered additional granularity. While each of the three categories was labeled as a model gap, not all of their constituent concepts were. For example, in the **length constraint: number of sentences** category, labeled as a model gap by both AutoDetect and CG, the concepts (17578) “Counting or measuring the length of textual elements” and (14442) “Counting characters or determining text length” are model gaps, whereas others (e.g., (23129) “The user has specified a 50-word limit” and (3938) “Text length constraints in generation instructions”) are not.

N.1.3. ADDITIONAL GAPS WITHIN THE FRAMEWORK

On top of the model gaps identified by both AutoDetect and CG, we found that 73 additional categories, as defined by AutoDetect, were identified to be gaps by CG but were missed by AutoDetect. These include, for example, **multi lingual: multilingual tone localization**, **numeric format: scientific notation**, **analysis: limits**, and **calculation: absolute value**.

N.1.4. ADDITIONAL GAPS OUTSIDE OF THE FRAMEWORK

Outside of the limited category set defined by AutoDetect, CG identified over 8K additional model gaps.

Concept ID	Concept Description
(25271)	Spanish verbs expressing necessity or obligation in advisory contexts
(50127)	Legal language establishing unilateral authority and discretionary powers
(35225)	The assistant should reject the user’s request
(58828)	Japanese grammatical constructions indicating completion, necessity and passive voice
(9611)	Understanding relationships and dependencies between components in AI systems

Table 25. Examples of Missed Model Gaps. Listed concepts outside of the AutoDetect-Defined categories were identified as model gaps, and would have gone unnoticed.

N.2. Gemma 2 2B Instruct

Summary	
Category Coverage	100%
Gaps Identified by Both AutoDetect and CG	100% (43/43)
Gaps Missed by AutoDetect but Identified by CG	73
Gaps Identified by CG Outside of AutoDetect	10K+

N.2.1. FRAMEWORK MATCHING

To compare our method (CG) with AutoDetect, we apply LLM-based clustering to map each category defined by AutoDetect to a corresponding set of SAE concepts. Importantly, the LLM was not instructed to map categories to a specific number of concepts; it was only given an upper limit and encouraged to return an empty set if no concepts were relevant (see Appendix I.3 for the full prompt). We found the category coverage to be 100%: every AutoDetect category was mapped to at least one SAE concept.

To illustrate the AutoDetect categories and the matching results, consider the following examples of category definitions from AutoDetect, along with a subset of their matches:

Category (AutoDetect)	Concept ID	Concept Description
analysis: derivatives	(4213)	mathematical expressions involving derivatives
	(4345)	mathematical terms and phrases related to derivatives and equations
	(10489)	mathematical expressions or calculations, particularly those related to derivatives and products
	(11880)	mathematical expressions and calculations related to derivatives and factors
	(15977)	mathematical expressions and functions involving derivatives
length constraint: summary	(2198)	elements indicating summaries, reflections, or clarifications
	(3685)	sections that summarize content or provide overviews
	(10511)	specific terms related to classification, guidelines, or categories
	(11863)	summaries and assessments of content
	(13671)	sentences that conclude or summarize points
mathematics and algorithms: algorithm design	(2042)	technical terms and references related to algorithms and computational processes
	(5574)	technical terminology related to algorithms and data processing
	(8411)	technical terms related to algorithm design and performance evaluation
	(9663)	mathematical terms related to computational problem-solving and algorithms
	(11271)	words and phrases related to improving and refining processes or methodologies

Table 26. Examples of AutoDetect Category Matches. Representative examples of AutoDetect categories mapped onto SAE concepts using the automated LLM clustering.

N.2.2. GAPS IDENTIFIED BY AUTODETECT

CG recovered all 43/43 (100%) model gaps identified by AutoDetect. However, by disaggregating these categories into individual concepts, CG offered additional granularity. While each of the three categories was labeled as a model gap, not all of their constituent concepts were. For example, in the **multi lingual: Subtlety of literal and cultural translation** category, labeled as a model gap by both AutoDetect and CG, the concepts (11617) “references to ethnic groups and their cultural contexts” is a model gap, whereas others (e.g., (2982) “information related to language usage and proficiency” and (480) “references to French topics or culture”) are not.

N.2.3. ADDITIONAL GAPS WITHIN THE FRAMEWORK

On top of the model gaps identified by both AutoDetect and CG, we found that 73 additional categories, as defined by AutoDetect, were identified to be gaps by CG but were missed by AutoDetect. These include, for example, **multi lingual: bilingual constraints**, **mathematics and algorithms: basic mathematical operations**, and **numeric format: scientific notation**.

N.2.4. ADDITIONAL GAPS OUTSIDE OF THE FRAMEWORK

Outside of the limited category set defined by AutoDetect, CG identified over 10K additional model gaps.

Concept ID	Concept Description
(5022)	code snippets related to phone number formatting and manipulation
(4087)	terms related to null or empty values in programming contexts
(1580)	formatting and layout commands in document typesetting
(13870)	Java Swing library components and related classes
(2278)	LaTeX commands or symbols used in mathematical formulations

Table 27. Examples of Missed Model Gaps. Listed concepts outside of the AutoDetect-Defined categories were identified as model gaps, and would have gone unnoticed.

O. Comparison with Other Methods: Arena-Hard-Auto

Arena-Hard-Auto (AHA) is a benchmark evaluation framework proposed by Li et al. (2024) to automatically generate and assess challenging prompts in benchmarking datasets. Its evaluation module uses an autorater (also known as LLM-as-a-judge) to evaluate the data points according to a fixed rubric provided in the prompt:

Your task is to evaluate how well the following input prompts can assess the capabilities of advanced AI assistants. For the input prompt, please analyze it based on the following 7 criteria.

1. Specificity: Does the prompt ask for a specific, well-defined output without leaving any ambiguity? This allows the AI to demonstrate its ability to follow instructions and generate a precise, targeted response.

2. Domain Knowledge: Does the prompt test the AI’s knowledge and understanding in a specific domain or set of domains? The prompt must demand the AI to have a strong prior knowledge or mastery of domain-specific concepts, theories, or principles.

3. Complexity: Does the prompt have multiple components, variables, or levels of depth and nuance? This assesses the AI’s capability to handle complex, multi-faceted problems beyond simple queries.

4. Problem-Solving: Does the prompt require active problem-solving: analyzing and clearly defining the problem and systematically devising and implementing a solution? Note active problem-solving is not simply reciting facts or following a fixed set of instructions.

5. Creativity: Does the prompt require a creative approach or solution? This tests the AI’s ability to generate novel ideas tailored to the specific needs of the request or problem at hand.

6. Technical Accuracy: Does the prompt require an answer with a high degree of technical accuracy, correctness and precision? This assesses the reliability and truthfulness of the AI’s outputs.

7. Real-World Application: Does the prompt relate to real-world applications? This tests the AI’s ability to provide practical and actionable information that could be implemented in real-life scenarios.

After analyzing the input prompt based on these criteria, you must list the criteria numbers that the prompt satisfies in the format of a Python array. For example, "[1, 2, 4, 6, 7]".

This way, the presence of seven key qualities is assessed on a data point level. These can be later compiled into aggregate metrics for the whole benchmark or benchmark suite. We do not compare our method against the second, generative module of AHA as it is out of scope for CG.

AHA does not have a visualization mechanism built in. It is interfaced through a command-line interface (CLI).

Matching. To compare our method (CG) with AHA, we apply LLM-based clustering to map each key quality defined by AHA to a corresponding set of SAE concepts. Importantly, the LLM was not instructed to map categories to a specific number of concepts; it was only given an upper limit and encouraged to return an empty set if no concepts were relevant (see Appendix I.3 for the full prompt). The coverage was 7/7 (100%): every key quality in AHA was mapped to at least one SAE concept.

CG Setup. For the purposes of this comparison, we employ only the Llama 3.1 8B model’s SAE.

O.1. AGI Eval EN

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		avg.	min.	max.	
1 specificity	1.00	0.05	0.00	0.45	24
2 domain knowledge	0.98	0.04	0.00	0.41	38
3 complexity	0.83	0.04	0.00	0.51	25
4 problem-solving	0.98	0.04	0.00	0.45	23
5 creativity	0.11	0.03	0.00	0.33	38
6 technical accuracy	1.00	0.07	0.00	1.23	31
7 real-world application	0.47	0.02	0.00	0.24	34

Table 28. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): AGI Eval EN. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.2. BBQ

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		avg.	min.	max.	
1 specificity	1.00	0.13	0.00	1.07	58
2 domain knowledge	0.80	0.06	0.00	0.25	70
3 complexity	0.14	0.08	0.00	0.54	57
4 problem-solving	0.78	0.02	0.00	0.11	62
5 creativity	0.01	0.22	0.00	1.97	55
6 technical accuracy	0.69	0.13	0.00	1.34	67
7 real-world application	0.23	0.04	0.00	0.22	60

Table 29. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): BBQ. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.3. CROWS Pairs

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.03	0.00	0.75	33
2 domain knowledge	1.00	0.01	0.00	0.08	49
3 complexity	0.57	0.03	0.00	0.69	42
4 problem-solving	1.00	0.01	0.00	0.06	41
5 creativity	0.99	0.02	0.00	0.56	23
6 technical accuracy	0.98	0.02	0.00	0.24	57
7 real-world application	0.98	0.01	0.00	0.10	40

Table 30. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): CROWS Pairs. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.4. GSM8K

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.04	0.00	0.38	28
2 domain knowledge	0.77	0.03	0.00	0.54	37
3 complexity	0.42	0.04	0.00	0.84	32
4 problem-solving	0.99	0.03	0.00	0.26	30
5 creativity	0.00	0.06	0.00	1.10	37
6 technical accuracy	0.99	0.04	0.00	0.87	34
7 real-world application	0.21	0.02	0.00	0.32	31

Table 31. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): GSM8K. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.5. LogicBench

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		avg.	min.	max.	
1 specificity	1.00	0.07	0.00	0.80	23
2 domain knowledge	0.84	0.10	0.00	1.86	40
3 complexity	0.57	0.06	0.00	1.34	23
4 problem-solving	0.98	0.05	0.00	1.19	26
5 creativity	0.04	0.05	0.00	0.47	15
6 technical accuracy	0.92	0.10	0.00	1.67	43
7 real-world application	0.26	0.05	0.00	0.61	27

Table 32. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): LogicBench. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.6. MATH

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		avg.	min.	max.	
1 specificity	1.00	0.02	0.00	0.28	16
2 domain knowledge	0.95	0.06	0.00	0.86	29
3 complexity	0.69	0.04	0.00	0.45	25
4 problem-solving	0.89	0.04	0.00	0.59	27
5 creativity	0.00	0.03	0.00	0.33	31
6 technical accuracy	1.00	0.07	0.00	1.14	29
7 real-world application	0.11	0.02	0.00	0.28	39

Table 33. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): MATH. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.7. Natural Questions

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.01	0.00	0.08	51
2 domain knowledge	0.95	0.01	0.00	0.09	50
3 complexity	0.01	0.01	0.00	0.05	58
4 problem-solving	0.18	0.01	0.00	0.04	48
5 creativity	0.01	0.01	0.00	0.10	43
6 technical accuracy	0.93	0.01	0.00	0.05	59
7 real-world application	0.17	0.00	0.00	0.07	46

Table 34. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): Natural Questions. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.8. Real Toxicity

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.01	0.00	0.21	33
2 domain knowledge	0.73	0.00	0.00	0.04	36
3 complexity	0.23	0.01	0.00	0.11	39
4 problem-solving	0.70	0.00	0.00	0.02	38
5 creativity	0.29	0.01	0.00	0.08	34
6 technical accuracy	0.55	0.00	0.00	0.04	41
7 real-world application	0.40	0.00	0.00	0.05	27

Table 35. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): Real Toxicity. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.9. Social IQA

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.02	0.00	0.36	44
2 domain knowledge	0.66	0.01	0.00	0.14	53
3 complexity	0.40	0.02	0.00	0.54	47
4 problem-solving	0.89	0.01	0.00	0.12	44
5 creativity	0.15	0.04	0.00	0.91	36
6 technical accuracy	0.28	0.01	0.00	0.11	64
7 real-world application	0.52	0.01	0.00	0.06	37

Table 36. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): Social IQA. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.10. Vectara

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		avg.	min.	max.	
1 specificity	1.00	0.61	0.00	5.58	7
2 domain knowledge	0.99	0.78	0.00	17.71	18
3 complexity	0.21	0.70	0.00	8.19	5
4 problem-solving	0.99	0.38	0.00	3.31	9
5 creativity	0.01	0.98	0.00	13.62	10
6 technical accuracy	1.00	0.66	0.00	12.91	22
7 real-world application	0.73	0.51	0.00	5.45	5

Table 37. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): Vectara. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

O.11. WinoGrande

AHA Category	AHA Score	X_{bench}			# Bench. Gaps
		<i>avg.</i>	<i>min.</i>	<i>max.</i>	
1 specificity	1.00	0.02	0.00	0.31	38
2 domain knowledge	0.12	0.01	0.00	0.16	50
3 complexity	0.00	0.03	0.00	0.77	47
4 problem-solving	1.00	0.00	0.00	0.03	39
5 creativity	0.01	0.03	0.00	0.70	37
6 technical accuracy	0.61	0.01	0.00	0.28	51
7 real-world application	0.64	0.01	0.00	0.08	37

Table 38. Arena-Hard-Auto (AHA) vs. Competency Gaps (CG): WinoGrande. The X_{bench} statistics are reported for all SAE concepts matched with the corresponding AHA category. The # *Bench. Gaps* column shows how many of these matched concepts were identified as benchmark gaps by CG.

P. Comparison with Other Methods: EvalTree

EvalTree is a framework proposed by Zeng et al. (2025) for profiling LLM weaknesses on a given benchmark. Given a benchmark dataset, EvalTree first labels each instance with a natural-language capability description, embeds and recursively clusters the instances to construct a hierarchical *capability tree*, and then computes per-subtree accuracy with confidence intervals. Subtrees whose upper-confidence accuracy falls below a chosen threshold are flagged as *weakness profiles*, each described by a natural-language phrase summarizing the cluster.

EvalTree is launched through a command-line interface (CLI), without a graphical interface. Its outputs include a JSON capability tree per benchmark and one weakness-profile list per significance threshold, stored as JSON files.

CG Setup. For this comparison, we reproduce EvalTree’s MATH baseline on the five language models analyzed in this paper. Following the EvalTree configuration in Zeng et al. (2025), we use the published gpt-4o-mini annotator, text-embedding-3-small embeddings, max-children=10 for the recursive clustering, and the default significance level $\alpha = 0.05$ with direction set to *lower*. For each model, we tabulate the number of weakness profiles returned across thresholds and inspect the most stringent profile (index 0) qualitatively.

P.1. MATH

Table 39 summarizes the EvalTree baseline on MATH for the five language models we analyzed. We report each model’s overall MATH accuracy and the number of weakness profiles returned by EvalTree across significance thresholds. Table 40 reports an example weakness profile per model (the first entry from the index-0 profile).

Table 39. EvalTree Baseline Summary on MATH. Overall MATH accuracy and number of weakness profiles surfaced by EvalTree (across thresholds, $\alpha = 0.05$, direction = lower) for the five models analyzed.

Model	MATH Acc.	Weakness Profiles
Gemma 2 2B	5.3%	4
Mistral 7B Instruct v0.1	7.7%	25
DeepSeek-R1-Distill-Llama 8B	47.1%	57
Qwen3-4B	38.9%	65
Llama 3.1 8B Instruct	42.7%	71

Table 40. Representative EvalTree Weakness on MATH. The first weakness profile (index 0, most stringent threshold) returned by EvalTree for each model.

Model	EvalTree Weakness Profile (representative)
Gemma 2 2B	Analyzing and applying mathematical functions and data to optimize relationships, interpret behaviors, and determine outcomes.
Mistral 7B Instruct v0.1	Analyzing and applying geometric and trigonometric principles to solve for angles, side lengths, and relationships in polygons and triangles.
DeepSeek-R1-Distill-Llama 8B	Analyzing and applying Vieta’s formulas and polynomial techniques to evaluate and understand the properties of polynomial roots.
Qwen3-4B	Analyzing and applying advanced mathematical techniques to evaluate functions, derive patterns, and manipulate sequences and series through recursive reasoning and geometric modeling.
Llama 3.1 8B Instruct	Analyzing and applying advanced mathematical techniques to evaluate functions, derive patterns, and manipulate sequences and series through recursive reasoning and geometric modeling.

CG vs. EvalTree on the Same Model. The two methods operate at very different granularities. EvalTree’s weakness profiles are skill-level natural-language phrases derived from clustering benchmark instances, while CG’s concepts are atomic axes from the model’s own SAE space. Table 41 illustrates this contrast on DeepSeek-R1-Distill-Llama 8B: EvalTree flags broad skill clusters (e.g., *Vieta’s formulas and polynomial techniques*), whereas CG surfaces concrete sub-components — the *form* the model handles well (LaTeX math code, reasoning-step transitions) versus the *substance* it handles poorly

(boxed numerical answers, multi-digit arithmetic). The two views are complementary: EvalTree organizes a benchmark’s instances into a digestible hierarchy of capability clusters, while CG decomposes performance into model-internal axes that generalize across benchmarks.

Table 41. EvalTree vs. CG (DeepSeek-R1-Distill-Llama 8B). Side-by-side illustration of the two methods’ weakness descriptions, drawn from the EvalTree weakness profile (index 0) and from CG’s worst-performing concepts on the same model (Table 7). CG concepts are more atomic and benchmark-agnostic; EvalTree clusters are coarser, benchmark-grounded skills.

EvalTree weakness profile	CG worst-performing concept
Analyzing and applying Vieta’s formulas and polynomial techniques to evaluate and understand the properties of polynomial roots.	(32637) boxed numerical answers in a mathematical context
Analyzing and applying vector operations and matrix transformations in multi-dimensional spaces.	(24736) multi-digit numbers
Manipulating and solving systems of linear equations in multiple dimensions through matrix operations and parameterization.	(9878) code blocks in C# with curly braces